Interop Tokyo 2026　配布資料

2026 年 6 月 | 幕張メッセ

Chromium 全 API バインディングの意味と意義

組み込み Web 描画エンジン実装の設計論

BlinkGTK Development Team

2026 年 4 月 17 日　初版

Creative Commons Attribution 4.0 International (CC BY 4.0)

目次

1. 前提: Web 描画エンジンの選択肢 4

2.「API バインディング」の定義と層構造 8

3.「全 API をバインドする」ことの技術的意味 15

4. 意義 —どのような応用で価値が顕現するか 21

5. WebGPU と AI 推論—新しい計算基盤の活用 36

6. 代替手段との比較 41

7. 取り組む際のコストと制約 45

8. 向き不向きの判断基準 52

9. W3C の貢献 —外部実装者の視点から 56

10. 結び 63

参考文献・リソース 66

要旨

本文書は、Chromium および Chromium 上に構築されるレンダリングスタック(Blink、V8、Skia、cc、Viz、Ozone 等)の API 群を、抽象化ラッパーを介さずに直接バインディングすることの技術的意味と、組み込み用途における意義を論じる。世の中には Web 描画エンジンを組み込む手段がすでに複数存在する。プラットフォーム付属の WebView、CEF や Electron のようなラッパーフレームワーク、WebKitGTK や Qt WebEngine のような他エンジンバインディング、Servo や Ladybird のような独立実装がそれである。それぞれに固有の設計思想と抽象化レベルがあり、適した用途も異なる。

本稿が主張するのは、**「Chromium の内部 API 層に至るまで余すところなくバインディングを提供する」**というアプローチは、一見すると遠大かつ過剰に見えるが、組み込みや特定用途向けの描画エンジン実装においては、他の手段では得がたい自由度と制御性をもたらす、ということである。特に電子書籍端末、キオスク、車載 HMI、医療画像、金融取引端末のように、単なる Web ページ表示にとどまらず、描画・入力・セキュリティ・データフローの各層に対する細やかな介入を必要とする領域において、全層バインディングは単なるオプションではなく、要求仕様を満たすための必須要件となる。

同時に本稿は、このアプローチが抱える現実的コストと制約についても誠実に論じる。Chromium の内部 API は公式に安定したものではなく、アップストリームの追随は継続的負担を伴う。ソースツリーの規模、ビルド時間、メモリフットプリント、求められる人的リソースのいずれも、軽量な代替手段と比較すれば大きい。本稿の結論は「全層バインディングはすべての用途に推奨される」ではなく、「どのような場合に有効で、どのような場合には向かないか、その判断基準を明確にする」ことにある。

なお本稿で論じるあらゆる応用領域 — 電子書籍からロボティクス、農業 IT、自動運転、エッジ AI に至るまで — は、W3C と WHATWG が四半世紀以上かけて策定してきた Web 標準群を土台とする。外部実装者として W3C の貢献への謝意を第 9 章で改めて表明する。

Interop Tokyo 2026 での配布にあたって

本文書は Interop Tokyo 2026 の配布資料として提供される。Interop ShowNet が 1994 年の Interop Japan 発足以来、「異なるベンダー機器の相互接続性を現場で実証する」という哲学を貫いてきたのと同様に、Web プラットフォームもまた、異なる実装間の相互運用性を至上の価値として歩んできた。ネット

ワーク層では TCP/IP・HTTP・TLS といったプロトコルの相互運用性が、Web プラットフォーム層では HTML・CSS・JavaScript・WebGPU・WebNN といった API の相互運用性が、いずれも膨大な標準化努力と実装検証によって支えられている。本文書の論旨は、この二つの世界の思想的親和性の上に成立している。ShowNet が複数ベンダー機器を実際に繋いで動くことを毎年示し続けるように、W3C と各ブラウザベンダーは Web Platform Tests という形で「実装の相互運用性」を示し続けている。Interop Tokyo 2026 の参加者の皆様にとって、本文書がネットワーク・インフラと Web プラットフォームの接点を考える一助となれば幸いである。

1. 前提: Web 描画エンジンの選択肢

組み込み用途で Web 描画エンジンを利用する際、現在の選択肢は大別して五系統に整理できる。本章では各系統の設計思想と抽象化レベルを俯瞰し、本稿が論ずる「直接バインディング」という選択肢の位置づけを明確にする。

ここで重要な前提を一つ確認しておく。以下で論じるすべての選択肢は、**W3C（およびこれと連携する WHATWG、IETF、Khronos Group 等）が策定してきた Web 標準群**という共通基盤に立脚している。HTML、CSS、DOM、Fetch、WebRTC、WebGL、WebGPU、Service Worker、WebXR、WebNN、Web of Things といった仕様群なしには、どのエンジンも、どのラッパーも、どの独立実装も存在しえない。Web プラットフォームが 30 年以上にわたり維持されてきたという単純な事実そのものが、W3C の積み重ねた成果である。本稿は Chromium という特定実装を中心に据えるが、論ずる内容のほとんどは W3C 標準に準拠する他実装にも同様に適用される。

1.1 独立実装 —Servo、Ladybird

Mozilla Research から派生し現在は Linux Foundation 傘下の Servo Project として開発が続く Servo、そして 2022 年に登場し独自 Web 標準実装を目指す Ladybird は、いずれも「Web エンジンをゼロから書き直す」というアプローチを取る。Servo は Rust で記述され、並列レンダリングや安全なメモリモデルを強みとする。Ladybird は C++ ベースで、SerenityOS から派生した独自コードベースを持つ。

これらの独立実装は魅力的な選択肢だが、Web 標準の広さは実装者にとって大きな障壁となる。HTML5、CSS3（Flexbox、Grid、Subgrid、Container Queries）、ECMAScript 2024、WebAssembly、Service Worker、IndexedDB、WebRTC、WebCodecs、WebGPU といった現代 Web プラットフォームが要求する機能群を網羅するには、数百人年規模の実装工数が必要となる。2026 年時点で Servo は特定サイトでの表示は可能だが、Google、YouTube、GitHub 等の主要サイトでの実用的表示には未だ課題がある。Ladybird も同様の状況にある。

したがって組み込み用途で実用的な Web 表示を求める場合、現実的にはすでに Web 標準を網羅的に実装しているエンジン（Blink、WebKit、Gecko）のいずれかを基盤とする必要がある。

1.2 プラットフォーム付属 WebView

プラットフォームが提供する WebView コンポーネントは、最も手軽な選択肢である。macOS / iOS の `WKWebView`、Android の `WebView`、Windows の `WebView2`（Edge Chromium ベース）がこれにあたる。

プラットフォーム WebView の長所は、エンジン本体の保守と更新をプラットフォームベンダーに委ねられることである。セキュリティパッチはシステム更新経由で適用され、アプリケーション開発者は API の呼び出しに集中できる。

しかし組み込み用途では次の制約が問題となる：

- **プラットフォーム依存**: Android WebView は Android 端末に、WKWebView は Apple プラットフォームに、WebView2 は Windows に縛られる。Linux ベースの組み込み機器や独自 OS では利用できない
- **API の制限**: 各プラットフォーム WebView はベンダーが定めた公開 API のみを提供する。内部のレンダリングパイプライン、Compositor 層、ネットワークスタックへの直接アクセスは不可能
- **更新サイクル**: プラットフォーム WebView はシステム更新に連動するため、組み込み機器のように長期間同一ファームウェアで運用する前提と整合しない
- **機能制御の不可能性**: たとえば「テキスト選択を禁止する」「コンテキストメニューを一切表示しない」「印刷機能を完全に無効化する」といったキオスク的要求に対して、公開 API では応えきれない場合が多い

1.3 ラッパー型 —CEF、Electron

Chromium Embedded Framework (CEF) は、Chromium をアプリケーションに組み込むための最も広く使われているラッパーライブラリである。C++ の API を提供し、Windows / macOS / Linux をサポートする。CEF は Chromium の `content/` レイヤーを抽象化し、`CefBrowserHost`、`CefRenderHandler`、`CefRequestHandler` 等のインターフェースとして再公開する。

Electron はこれをさらに発展させ、Chromium に Node.js ランタイムを統合してデスクトップアプリケーション開発の統合環境とした。Slack、Visual Studio Code、Discord、Figma、GitHub Desktop など主要アプリケーションで採用されている。

これらラッパー型の長所は、Chromium 特有の複雑さを隠蔽し、比較的安定した API でアプリケーション開発を可能にする点にある。CEF は開発開始からすでに 15 年以上の歴史があり、豊富なサンプルコードとドキュメントが存在する。

しかし「ラッパーで抽象化する」ということは、裏を返せば「ラッパーが露出しない層には手が届かない」ということでもある。CEF のソースコードを読めばわかるように、CEF は Chromium の `content::WebContents` や `content::RenderWidgetHostView` を内部で保持しているが、これらを直接操作する API は限定的にしか提供されていない。特定の挙動をカスタマイズしたい場合、CEF そのものにパッチを当てるか、CEF をフォークする必要が生じる。

また Electron は Node.js 統合による利便性と引き換えに、バンドルサイズ（通常 100MB 以上）、メモリ使用量（数百 MB 以上）、起動時間（数秒）のオーバーヘッドを伴う。組み込み機器の多くはメモリ 512MB〜2GB、ストレージ数 GB〜十数 GB という制約下にあり、Electron は適合しない。

1.4 他エンジンバインディング— WebKitGTK、Qt WebEngine

WebKitGTK は WebKit を GTK 向けにバインディングしたライブラリで、GNOME の Epiphany ブラウザや多数の GTK アプリケーションで利用されている。LGPL 2.1 ライセンスで、ソースコードレベルの改変は比較的容易である。

Qt WebEngine は Chromium の Blink エンジンを Qt フレームワークに統合したもので、Qt アプリケーションから `QWebEngineView` としてアクセスできる。Qt のシグナル・スロット機構に統合され、Qt エコシステムとの親和性が高い。

これらは「特定 UI フレームワークの流儀に合わせた Web エンジン統合」という位置づけで、それぞれのエコシステム内では優れた選択肢となる。しかし次の特性を理解しておく必要がある：

- **API は UI フレームワークの設計思想に従う**: たとえば WebKitGTK は `WebKitWebView` を `GtkWidget` として扱い、Qt WebEngine は `QWebEngineView` を `QWidget` として扱う。これ自体は正しい統合だが、各フレームワークの流儀で Web エンジンを「覆って」いるため、エンジン内部の挙動への介入には限界がある

- **エンジン本体のバージョン制御**: WebKitGTK は WebKit 本家の更新に追随し、Qt WebEngine は Chromium の更新に追随するが、その追随速度と選択されるバージョンはディストリビューション管理者に委ねられる場合が多い
- **カスタマイズは原則としてソース改変**: Chromium 内部の挙動を変更したい場合、Qt WebEngine であればカスタムビルドが必要となり、WebKitGTK も同様

1.5 直接バインディング—本稿のアプローチ

以上の選択肢と並ぶ五番目のアプローチが、本稿が論ずる「直接バインディング」である。これは次のように定義される：

> **Chromium のソースツリーそのものを含む形で自プロジェクトを構成し、`content::` 名前空間以下を含む内部 API に対して、UI フレームワーク（たとえば GTK4）向けの薄いバインディング層のみを挟む構成**

この定義には重要な含意がいくつかある：

1. **Chromium のバージョンを自プロジェクトが完全に制御する**。どの Chromium 安定版を基盤にし、いつ更新するかを決定できる
2. **Chromium 内部のソースコードへの変更が許容される**。必要に応じて Chromium 本体にパッチを当て、自プロジェクトで維持する
3. **`content::`、`cc::`、`viz::`、`ui/ozone/`、`ui/gl/`、`ui/base/ime/` 等の内部 API にも直接バインディングを提供する**。UI フレームワークの API として、ラッパーを介さず直接露出する
4. **抽象化層の最小化**。UI フレームワーク側が必要とする型変換（たとえば GTK4 の `GtkWidget` への接続）以外の抽象化は設けない

この定義により、本アプローチは前述の四選択肢のいずれとも異なる位置を占める。独立実装ほど遠大ではないが、プラットフォーム WebView より遥かに深い制御を可能とし、ラッパー型より露出面積が広く、他エンジンバインディングより Chromium 固有の機能への追随が速い。

本稿の残りの章では、この選択がどのような技術的意味を持ち、どのような応用場面で価値を発揮するか、そしてどのようなコストを伴うかを体系的に論ずる。

2.「API バインディング」の定義と層構造

「API をバインドする」と一口に言っても、Web 描画エンジンの API は単一層ではない。Chromium を例にとれば、最上層の「ページを読み込む」API から、最下層の「GPU バッファを Wayland コンポジタに渡す」API まで、少なくとも数十の層が存在する。本章ではこの層構造を整理し、各層でどのような API がバインド対象となるかを具体的に示す。

2.1 表層 API —アプリケーションが最初に触れる面

最も表層に位置するのは、「ブラウザのように使う」ための API である。具体的には次のような操作を提供する：

```
// 最小限の C API の例
BlinkWebView* view = blink_web_view_new();
blink_web_view_load_uri(view, "https://www.example.com");
blink_web_view_go_back(view);
blink_web_view_go_forward(view);
blink_web_view_reload(view);

const char* url = blink_web_view_get_uri(view);
const char* title = blink_web_view_get_title(view);
double progress = blink_web_view_get_estimated_load_progress(view);
```

この層は概念的には WebKitGTK の `WebKitWebView` や Qt WebEngine の `QWebEngineView` と対応する。どの実装を選んでもほぼ同等の API を提供する。

しかしこの層だけでは、組み込み用途の要求を満たすには不十分である。たとえば「このページで特定の DOM 要素を検査して、その存在に応じて処理を分岐したい」という要求には、JavaScript 実行と結果受信の API が必要となり、これは表層と内部層の境界に位置する。

2.2 Content 層 —Chromium の中核抽象

Chromium の `content/` ディレクトリは、ブラウザエンジンの中核概念を提供する。ここで定義される主要型は以下のとおり：

- **`content::BrowserContext`**: ブラウザのセッションを表す。Cookie、ストレージ、認証情報の永続化単位。Incognito モードの実装もこのレベルで分離される
- **`content::WebContents`**: 一つの Web ページを表す。タブブラウザにおける「タブ」に相当する抽象。ナビゲーション履歴、URL、タイトル、ファビコン、ロード状態を保持する
- **`content::RenderFrameHost`**: 一つのフレームを表す。トップフレームとサブフレーム(iframe)がそれぞれ RFH として管理される
- **`content::RenderWidgetHost`**: 一つの描画領域を表す。通常 RFH と対になって存在し、入力イベントの受け口でもある
- **`content::RenderWidgetHostView`**: RenderWidgetHost の UI 側の表現。プラットフォーム依存のウィジェット(`RenderWidgetHostViewAura`、`RenderWidgetHostViewMac` 等)として実装される

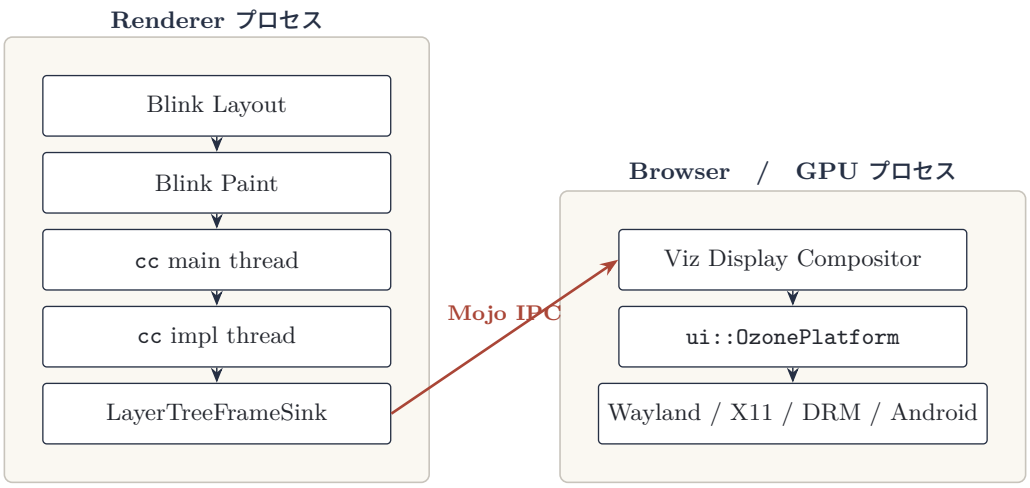
これらの型群に対するバインディングを提供することで、アプリケーションは次のような介入が可能となる：

- 特定 URL のナビゲーションをブロックする
- 新規ウィンドウ要求(`target="_blank"`、`window.open()`)を傍受する
- JavaScript ダイアログ(`alert()`、`confirm()`、`prompt()`)をカスタム UI で置き換える
- ファイル選択ダイアログ(`<input type="file">`)をアプリケーション側で処理する
- リソース読み込み(画像、スクリプト、XHR)を個別にフィルタする
- SSL 証明書エラーの扱いを決定する

これらはいずれも、`WebContentsDelegate`、`WebContentsObserver`、`NavigationThrottle` といった Chromium の拡張点(extension point)を通じて実装される。拡張点の数は Chromium のソースコード上で百を優に超え、それぞれがバインディング対象となりうる。

2.3 Compositor / GPU 層 —描画パイプライン

画面に実際のピクセルを出す経路は、Chromium の中でも特に複雑な領域である。大まかな流れは、次の図のように二つのプロセスにまたがる。



レンダラープロセス側では、Blink Layout（レイアウト決定）から始まり、Blink Paint（描画命令生成）、cc（Chromium Compositor）のメインスレッド処理、cc の impl スレッド処理を経て、最終的に LayerTreeFrameSink が viz::CompositorFrame（描画指示の集合）を生成する。これが Mojo IPC を介してブラウザ／GPU プロセスへ送出され、Viz Display Compositor で合成されたのち、ui::OzonePlatform の抽象化レイヤーを通じて Wayland、X11、DRM、Android SurfaceFlinger といったプラットフォーム固有の描画経路に書き出される。

この経路には次の主要構成要素が登場する：

- **cc::LayerTreeHost**: レンダラープロセス側でレイヤツリーを管理する
- **cc::LayerTreeFrameSink**: レイヤツリーから viz::CompositorFrame を生成し、Viz に送出する
- **viz::CompositorFrameSinkSupport**: Viz 側で CompositorFrame を受け取り、ディスプレイコンポジタに渡す
- **viz::Display**: 最終的なフレームを合成し、出力デバイスに書き出す
- **ui::OzonePlatform**: Wayland、X11、DRM 等のプラットフォーム抽象化
- **ui::Compositor**: ブラウザプロセス側でウィンドウ単位のコンポジタを管理する

組み込み用途で GPU 層への介入が必要となる典型例は次のようなものである：

1. **dmabuf 共有メモリ経由のフレーム受け取り**: レンダラーで生成された GPU テクスチャを、zero-copy で GTK の `GdkDmabufTexture` に渡す
2. **カスタムフレームレート制御**: デフォルトの 60 fps ではなく、特定 fps（例: 30 fps 省電力モード、120 fps 高応答モード）に制御
3. **オフスクリーンレンダリング（OSR）**：ネイティブウィンドウを持たず、ビットマップとして出力を取得する
4. **複数ウィンドウ出力**: 一つの WebContents の出力を、複数のネイティブウィンドウに同期して表示する
5. **タイル化ストリーミング**: ページ全体を一度に描画せず、表示領域のタイルのみを要求する

これらはいずれも、`cc::LayerTreeHostClient`、`viz::DisplayClient`、`ui::PlatformWindow` 等のインターフェースを実装することで実現される。これらインターフェースの総メソッド数は数十に達し、それぞれの呼び出しタイミング、スレッド制約、ライフタイム要求を正しく理解する必要がある。

2.4 Input 層 —入力イベントのルーティング

ユーザー入力がブラウザエンジンに届くまでの経路も、単純な転送ではない。次のような段階を順に経る。

1. OS または Wayland コンポジタから入力イベント（マウス移動、ボタン押下、キー押下、タッチ等）が生成される
2. UI フレームワーク（GTK4 等）がこれを受け取り、言語やフォーカス状態を反映したイベントオブジェクトに変換する
3. アプリケーション層がそれを受信し、Chromium 向けの形式に再変換する
4. `blink::WebMouseEvent` / `blink::WebKeyboardEvent` / `blink::WebTouchEvent` 等の型に包まれる
5. `content::RenderWidgetHost::ForwardMouseEvent()` 等のメソッドで Mojo IPC を介してレンダラープロセスに送信される

6. レンダラープロセス側で Blink がイベントをディスパッチし、対応する DOM ターゲットを決定する
7. 最後に JavaScript イベントハンドラや CSS の `:hover` 等のスタイル変化が発火する

日本語・中国語・韓国語の入力には IME（Input Method Editor）の介在があり、これは更に複雑な経路を辿る：

- **GTK 側**: `GtkIMContext` がプリエディット文字列の変換を担当
- **Chromium 側**: `ui::InputMethod`、`ui::TextInputClient` が IME 状態を管理
- **Renderer**: `blink::InputMethodController` が Web ページに IME 入力を届ける

組み込み用途では次のような要求が生じうる：

- 物理キーボードが無い機器でソフトウェアキーボードを連動させる
- タッチパネルのジェスチャを独自定義（たとえば二本指スワイプをページ移動に割り当てる）
- 特定キー（F1〜F12、電源ボタン等）をアプリケーション側で傍受し、ブラウザに届けない
- バーコードリーダーや RFID リーダー等の HID デバイス入力を、特定の入力フィールドに直接注入する
- 車載環境でステアリングホイールのボタンを Web ページのナビゲーションに連動させる

これらは全層バインディングでなければ実装できない、あるいは実装が極めて煩雑になる機能である。

2.5 セキュリティ層

Chromium のセキュリティモデルは、現代のブラウザの中でも最も精緻な部類に属する。主要構成要素は次のとおり：

- **Process Sandbox**: レンダラープロセスを seccomp-bpf、namespace、setuid sandbox で隔離
- **Site Isolation**: 異なるオリジンを異なるプロセスに分離
- **Same-Origin Policy**: JavaScript と CSS からのクロスオリジンアクセスを制限
- **Content Security Policy (CSP)**: ページが宣言したポリシーに従いリソースをフィルタ

- **Mixed Content**: HTTPS ページから HTTP リソースを取得することを制限
- **SSL/TLS 証明書検証**: `net::CertVerifier` が証明書チェーンを検証
- **HSTS (HTTP Strict Transport Security)**: HTTPS 強制
- **Certificate Transparency**: 証明書の公開ログへの記録を要求

組み込み用途で調整が必要となる典型例：

1. **独自 CA の信頼**: 企業内 CA や組み込み機器メーカーの CA を信頼ストアに追加
2. **SSL エラーの自動処理**: 特定のエラーを自動的に受け入れる、または拒否する
3. **Sandbox の制約緩和**: 特定のシステムコール(たとえば組み込み機器固有のデバイスアクセス)を許可
4. **オリジン制約の調整**: 同一オリジンポリシーを緩和して、複数サブドメイン間の通信を許可
5. **証明書ピンニング**: 特定ホストに対して特定証明書のみを受け入れる

セキュリティ層の調整は、軽々に行うと重大な脆弱性を生むが、組み込み機器の固有環境では必要となる場合がある。これらは `content::ContentBrowserClient::AllowCertificateError()`、`content::BrowserPolicyConnector`、`net::URLRequestContextBuilder` 等を通じて実装される。

2.6 Media / Network / Storage 層

これらの層は Web プラットフォームの基盤機能を提供する：

Media 層: - **Media Source Extensions (MSE)**: ストリーミング再生の基盤 - **Encrypted Media Extensions (EME)**: DRM 付きコンテンツの再生 - **WebRTC**: P2P 通信とメディアストリーム- **WebCodecs**: 低レベルなコーデックアクセス - **Picture-in-Picture**: ピクチャインピクチャ表示

Network 層: - `net::URLRequestContext`: HTTP/HTTPS リクエストの実行文脈 - **Cookie ストア**: 永続的・セッション的 Cookie の管理 - **HTTP キャッシュ**: レスポンスのキャッシュ - **Proxy 設定**: PAC スクリプト、静的プロキシ、システム設定の読み取り - **QUIC / HTTP/2 / HTTP/3**: プロトコル実装 - **Socket レベル介入**: カスタムソケット実装

Storage 層: - **IndexedDB**: 構造化データの永続保存 - **localStorage / sessionStorage**: キーバリュー保存 - **Cache API**: Service Worker と連動するキャッシュ - **FileSystem API**: ローカルファイルシステムへのアクセス（オリジン分離） - **Quota Manager**: オリジンごとのストレージ容量管理

これらの層に対するバインディングを提供することで、次のような応用が可能となる：

- オフライン動作を前提とする組み込み機器で、ストレージ使用量を厳密に制御
- 特定ホストへのリクエストをローカルのリソースに置き換える（たとえば機器内蔵サーバーへのリダイレクト）
- DRM キーの取得を独自ハードウェアセキュリティモジュールと連携
- メディア再生の出力先を、通常のスピーカーではなく組み込み機器固有の音声出力に指向

3.「全 API をバインドする」ことの技術的意味

前章で示したように、Web 描画エンジンの API は極めて多層的で、各層に数多くの拡張点が存在する。「全 API をバインドする」と表現したとき、それが何を技術的に意味するかを本章で整理する。なお、文字通り「すべて」のシンボルを一つ残らず公開することは実際的ではない（Chromium の内部ヘルパー関数まで公開する必要はない）。ここで論ずるのは「必要なときに必要な層に介入できる、開かれた設計」の意味である。

3.1 半透過抽象（leaky abstraction）の回避

ソフトウェアエンジニアリングの古典的概念に「すべての抽象は漏れる (All non-trivial abstractions, to some degree, are leaky)」という Joel Spolsky の言明がある。これは Web 描画エンジンのバインディングにおいても極めて強く当てはまる。

ラッパー型ライブラリ（CEF や WebKitGTK）は、一定のモデルで Web エンジンを抽象化して提供する。このモデルは大多数のユースケースには適合するが、次のような場合に「漏れ」が生じる：

事例 1: カスタムプロトコルハンドラ

`chrome://` や `webkit://` のような独自スキームを登録したい場合、通常のラッパー API には `RegisterCustomSchemeHandler` のような関数が用意されている。しかしその関数が受け付けるパラメータ（たとえば「このスキームは Service Worker を起動するか」「CORS の対象となるか」「Mixed Content 警告を出すか」）は、ラッパー設計者が想定した範囲に限られる。想定外のスキーム属性を設定したい場合、ラッパーの API では表現できず、結局ラッパー自体を改変することになる。

事例 2: Renderer プロセスへのメッセージ注入

Blink のレンダラー内で特定の JavaScript オブジェクトを定義したい、あるいは特定のイベントを傍受したいといった要求は、Web エンジンの内部動作に深く踏み込む。Chromium では `content::RenderFrameObserver` や `blink::WebLocalFrameClient` を通じてこれが可能だが、多くのラッパーはこれらの拡張点を公開しない。

事例 3: 描画タイミングの制御

「ページ読み込み完了からちょうど 500 ミリ秒後にスクリーンショットを撮る」といった要求は、一見単純に見えるが正確な実装は難しい。`DOMContentLoaded`、`load`、`onVisuallyNonEmpty`、`FirstContentfulPaint`、`LargestContentfulPaint` 等、複数の「完了」イベントがある中で、どれを基準にするかが問題となる。全層バインディングであれば`content::WebContentsObserver::DidStopLoading()`や`content::DocumentOnLoadCompletedInPrimaryMainFrame()`、`cc::LayerTreeHostClient::DidCommitAndDrawF`等の細かいフックを直接利用できるが、抽象化されたラッパーでは「ページ読み込み完了」という単一イベントしか提供されず、用途に応じた選択が不可能となる。

全層バインディングは、これら「漏れ」を設計レベルで防ぐ。抽象化は必要に応じて呼び出し側が導入すればよく、バインディング層自体は Chromium の提供する粒度をそのまま露出する。

3.2 カスタマイズ可能性の確保

組み込み用途の要求は、汎用ブラウザの要求とは異なる場合が多い。全層バインディングは、各層に対する個別のカスタマイズを可能にする。

カスタマイズの具体例:

- **表層**: スプラッシュ画面を自前で実装し、ページ読み込み完了までネイティブ側で表示を保持
- **Content 層**: ナビゲーション時に認証トークンをヘッダに自動注入する `NavigationThrottle`
- **Compositor 層**: 省電力モード時に Commit 頻度を 60 Hz から 15 Hz に動的変更
- **Input 層**: 特定のタッチジェスチャを、Web ページではなくネイティブ UI にルーティング
- **セキュリティ層**: 社内 CA で署名されたコンテンツに対する特別な権限付与
- **Media 層**: 音声出力先を、ブラウザデフォルトではなく、機器固有のオーディオデバイスに切り替え

これらはいずれも「組み込み機器固有の要求」として典型的なものだが、汎用ブラウザ API では全てを表現することは不可能である。

3.3 パフォーマンス最適化の自由度

パフォーマンス最適化の多くは、「特定の処理を省略する」あるいは「特定のタイミングで特定のコードパスを選ぶ」という形を取る。これらは抽象化された API の裏では実施不可能、あるいは極めて困難となる。

最適化の典型例:

1. **非必要機能の無効化**
   - WebGL / WebGPU を使わない用途で GPU プロセス自体を起動しない
   - 印刷機能を使わない用途で印刷関連のコード経路を Linker で削除
   - WebRTC を使わない用途で音声キャプチャ初期化をスキップ
2. **リソース読み込みの前倒し**
   - 起動時に固定コンテンツをプリフェッチ
   - Service Worker の登録を強制
   - HTTP キャッシュを読み取り専用で初期化
3. **描画パイプラインの短絡**
   - GPU 合成を無効化し、直接 Software 合成経路を取る
   - CopyOutputRequest を使わず、EndPaint フックで直接ピクセルを取得
   - レイヤツリーのリセット頻度を下げる
4. **メモリ管理の最適化**
   - V8 ヒープの初期サイズ・上限を機器に合わせて調整
   - イメージデコードキャッシュのサイズ制限
   - タイルメモリ予算の厳格な上限設定

これらの最適化は、通常 10〜30% のメモリ削減や、組み込み機器では体感できるレベルの応答性向上をもたらす。しかし「API を呼び出す」だけでは実現できず、Chromium のビルド構成(`gn args`)やコマンドライン引数(`--in-process-gpu`、`--disable-gpu-compositing` 等)を細かく制御する必要がある。全層バインディングはこれを自プロジェクト内で完結させる。

3.4 最新機能への直接アクセス

Web プラットフォームは急速に進化し続けている。WebGPU は 2023 年に Chrome で安定化し、View Transitions API は 2024 年に一般ページに降りてきた。WebNN（Web Neural Network API）、Compression Streams、Origin Private File System、Scheduler API など、毎四半期のように新機能が追加される。

全層バインディングのプロジェクトは、Chromium の更新に追随することで、これら新機能を即座に利用できる。ラッパー型では、ラッパーライブラリの保守者が新機能を公開するまで利用できない。プラットフォーム WebView では、プラットフォームベンダーの更新サイクルに依存する。

Origin Trial への参加もこの文脈で重要である。Chromium は実験的機能を「Origin Trial」という仕組みで限定公開する。参加するには特別なトークンが必要で、`--enable-features=<FeatureName>` や `--origin-trial-public-key=<Key>` のようなフラグ操作が必要となる。全層バインディングであれば、これらを自由に設定できる。

3.5 IPC 経路の直接利用

Chromium のアーキテクチャはマルチプロセスである。ブラウザプロセス、レンダラープロセス、GPU プロセス、ユーティリティプロセス、ネットワークプロセスが、Mojo と呼ばれる IPC 機構で通信する。

応用によっては、この IPC 経路に直接介入したい場合がある：

- ブラウザプロセス →レンダラープロセスに独自メッセージを送る（たとえば親アプリから子ページへのデータ配信）
- レンダラープロセス →ブラウザプロセスに独自メッセージを送る（たとえばページ内イベントのアプリへの通知）
- GPU プロセスの処理をプロファイルして、特定パターンの最適化を加える

Mojo はインターフェース定義言語（`.mojom`）で通信コントラクトを定義し、コード生成器が C++ / Java / JavaScript 等のバインディングを作る。全層バインディングのプロジェクトでは、自プロジェクト固有の `.mojom` を追加し、プロセス間通信を設計することが可能である。

たとえば組み込み機器で「Web ページ内から機器固有センサー(温度、加速度、GPS)の値を取得したい」という要求を考える。Web 標準だけで実装するには、Sensor API や Generic Sensor API の範囲に限られる。しかしこれらは汎用 Web ブラウザを想定しており、組み込み機器固有のセンサーには対応しない。

Mojo を直接利用すれば、次のような設計が可能である。まずブラウザプロセス側に独自サービス(例: `EmbeddedDeviceService`)を実装し、`.mojom` ファイルでインターフェースを定義する(例: `interface EmbeddedDevice { GetTemperature() => (float celsius); }`)。次にレンダラー側に対応する WebIDL 定義を追加し、`window.blinkgtkDevice.getTemperature()` として JavaScript から呼び出せるようにする。JavaScript からこの API を呼ぶと、Mojo 経由でブラウザプロセスへ要求が伝達され、ブラウザプロセス側では OS 固有 API (Linux なら `/sys/class/thermal/thermal_zone0/temp` 等)を読み取って値を返す、という連携が成立する。

このような拡張は、ラッパー型ライブラリでは原則として不可能である。`.mojom` ファイルを追加し、コード生成のビルド規則を組み、Blink 側の bindings を定義するという一連の作業は、Chromium ビルドシステムの内部に手を入れることを意味するからである。

3.6 ビルド構成の完全な制御

全層バインディングが可能とするもう一つの重要な自由度は、Chromium のビルド構成そのものへの介入である。Chromium は GN (Generate Ninja)を用いた巨大なビルドシステムを持ち、数百のビルドフラグで機能の有効化・無効化を制御できる。

具体例:

- `enable_webgl = false` により WebGL 関連コード(数 MB)を完全に除去
- `enable_printing = false` により印刷関連機能(数百 KB)を除去
- `proprietary_codecs = false` により独占的コーデック(H.264、AAC)を除去
- `enable_nacl = false` により Native Client 関連を除去(Chromium 147 で既定 false)
- `safe_browsing_mode = 0` により Safe Browsing 連携を無効化
- `toolkit_views = false` 等でウィジェット実装を切り替え

これらのフラグ変更は、バイナリサイズを数十〜数百 MB 削減し、アタックサーフェスを縮小し、ビルド時間を短縮する。組み込み用途ではフラッシュ容量・メモリが制約となるため、こうした最適化が実質的な意味を持つ。

ラッパー型ライブラリは通常、上流が配布するバイナリを使うか、あるいは特定のビルド構成にフィックスされた再配布物を使う。ビルドフラグの変更には、ラッパーのビルドシステム自体に手を入れるか、Chromium 本体のビルドから行う必要があり、結局直接バインディングと同等の労力を要することになる。

4. 意義 ―どのような応用で価値が顕現するか

前章まで、全層バインディングの技術的意味を論じてきた。本章では、そうした技術的特性が具体的にどのような応用で価値を発揮するかを、六つの代表的な領域に分けて論ずる。各領域について、典型的な要求仕様と、それを満たすために必要な層への介入を具体的に示す。

4.1 電子書籍・教科書端末

電子書籍端末、デジタル教科書ビューア、電子図書館アプリケーションは、組み込み Web 技術の重要な応用領域である。EPUB 3 は HTML5 と CSS3 をコンテンツ形式として採用しており、Web エンジンで表示できる。

この領域の典型的要求：

コンテンツ保護 - テキストのコピーを完全に禁止する（`user-select: none` の JavaScript レベル強制）- 右クリックによるコンテキストメニューを無効化する - スクリーンショット撮影を抑止する（OS レベルの協力が必要）- 印刷機能を無効化、または特定ページのみ印刷可能に制限 - 開発者ツール（DevTools）へのアクセスを完全に遮断

表示制御 - 縦書き・縦中横・ルビ表示の正確な実装 - ページめくり時の描画を同期させる（カクつきの排除）- フォントのキャッシュを事前に温める - 画像のデコードを事前実行する

閲覧履歴・進捗管理 - 読了位置を OS ファイルシステムではなく機器内蔵の DRM モジュールに記録 - ページ遷移時刻を厳密に計測（学習ログ用途）- 特定ページ滞在時間を計測

これらの要求のうち、「コピー禁止」は Web 標準の CSS `user-select: none` だけでは不十分である。Chromium のクリップボード API、`ui::TextInputClient`、`blink::WebInputEvent` の全層に亘ってコピー経路を塞ぐ必要があり、これは全層バインディングでなければ困難である。

「開発者ツール遮断」も `chrome://inspect` 経由のリモートデバッグを含めて完全に塞ぐには、Chromium のコマンドラインスイッチとビルド構成、プロセスのコマンドライン検査を組み合わせる必要がある。

4.2 キオスク・デジタルサイネージ

空港の案内端末、コンビニエンスストアのセルフレジ、美術館の音声ガイド、駅構内の広告ディスプレイ等、「ユーザーに特定のコンテンツだけを表示する端末」は広範に存在する。

この領域の典型的要求：

起動・動作の固定化 - 起動時に特定 URL を自動ロード - セッションタイムアウト後に初期 URL にリセット - ナビゲーションを特定ドメイン内のみに制限（外部サイトへの遷移をブロック） - 前進・後退ボタン、アドレスバー等のブラウザ UI を一切表示しない

操作制限 - 右クリック、長押しメニュー、テキスト選択を禁止 - キーボードショートカット（Ctrl+T、Ctrl+N、F11 等）を無効化 - ピンチズーム、ダブルタップズームを無効化 - 画面全体を固定し、フォーカスを特定領域から外れないようにする

セキュリティ - OS のファイルシステム、外部 USB、ネットワーク共有へのアクセスを遮断 - カメラ、マイクへのアクセスを明示的に許可したサイトのみに制限 - Cookie、localStorage を再起動時にクリア - 印刷機能を無効化

運用 - クラッシュ時の自動復旧（watchdog 連携） - 画面ロック中も定期更新を継続 - 画面焼き付き防止のための微小な画面振動 - 遠隔管理プロトコルによる設定更新

キオスク用途で特に重要なのは「Web ページが何をしようともユーザーに制御を渡さない」ことである。`window.open()` を呼ばれても新規ウィンドウを開かせない、`document.exitFullscreen()` を呼ばれても全画面表示を維持する、`location.href = "..."` で外部サイトに飛ばされてもブロックする、といった要求は、全てを Web ページのソース管理で解決できない以上、ブラウザ側の強制が必要となる。

4.3 車載 HMI・産業機器

自動車のセンターコンソール（IVI: In-Vehicle Infotainment）、工場の制御盤、医療機器の操作インターフェース、ATM、券売機、複合機のタッチパネル等、「組み込みコンピュータに Web 技術で UI を載せる」用途である。

この領域の典型的要求：

応答性 - タッチ入力からレスポンスまでの遅延を 50 ミリ秒以下に - アニメーションを常時 60 fps（あるいは 120 fps）で維持 - 温度・CPU 負荷に応じた品質調整

リソース制約 - RAM 512 MB〜2 GB、Flash 4 GB〜16 GB の環境での動作 - 起動から最初の画面表示まで 5 秒以内 - GPU は組み込み向け統合 GPU（Mali、Adreno、Intel UHD）

入力デバイス統合 - ステアリングホイールのボタン、物理ダイヤル、ジョイスティック等の入力を Web ページに届ける - タッチパネル以外の入力を、Web ページのフォーカス状態に応じて振り分け - 音声入力を、Web ページのマイク入力ではなく OS レベルで処理して結果をテキストとして届ける

認証・セキュリティ - 工場出荷時に焼き込まれた証明書による認証 - 特定のネットワーク（CAN bus、工場内 LAN）との連携 - 改竄検出（ファーム署名、ランタイム整合性チェック）

この領域では「Web エンジンを使う」ことは目的ではなく手段である。UI の開発生産性、国際化対応、HTML/CSS の柔軟性が目的であり、それを組み込み機器の厳しい制約下で実現する必要がある。全層バインディングでなければ、応答性の最適化やリソース制約への適合は実現困難である。

4.4 金融取引端末

証券会社のトレーディング端末、銀行の行員用端末、暗号通貨取引所のプロフェッショナル画面等、「低遅延・高信頼性の取引操作を Web UI で行う」用途である。

この領域の典型的要求：

低遅延 - WebSocket 経由の価格更新を、画面反映まで 10 ミリ秒以下で- キーボード・マウス入力から発注 API 呼び出しまで 5 ミリ秒以下で - 描画パイプラインのオーバーヘッドを最小化

データ整合性 - 画面表示と実際のデータの不一致を検出 - 更新順序の保証（後から来たデータで古いデータが上書きされないこと）- 部分的な更新失敗時の明示的エラー表示

セキュリティ - 画面コピーの抑止（重要データのスクリーンショット防止）- 入力データの暗号化（キーロガーへの耐性）- 操作ログの改竄防止

監査 - 全ての操作を定量的に記録 - 画面表示内容の定期的なスナップショット - 異常動作の検知（応答遅延、メモリリーク）

金融端末で特徴的なのは「見えているものと実際のデータの整合性」への厳しい要求である。Web ページが何らかの理由で古いデータを表示している場合(たとえば描画が数フレーム遅れている、CSS transition 中で最新データが見えていない等)、その事実をアプリケーション側で検知する必要がある。これには `cc::LayerTreeHostClient::DidCommit()` や描画ピクセルの直接取得(`CopyFromSurface`)といった Compositor 層への介入が必要となる。

4.5 医療画像ビューア・診断支援

放射線科の画像読影、病理診断、遠隔診療の画面表示、歯科用 X 線ビューア等、「医療画像を正確に表示し、診断者の判断を支援する」用途である。

この領域の典型的要求:

色精度 - キャリブレーション済みディスプレイとの連携(ICC プロファイル適用) - 医療画像の DICOM タグに準拠した輝度・コントラスト - グレースケール画像の 10 ビット・12 ビット表示

解像度 - HiDPI (200% 以上)の正確な表示 - 画像の拡大・縮小時の補間アルゴリズム選択(Lanczos、Bicubic 等) - 高解像度ディスプレイ(4K、8K)の十分な活用

ワークフロー - PACS (画像保管通信システム)との連携 - DICOM ネットワークプロトコルの実装 - HL7 FHIR によるデータ交換

安全性 - 診断に使う画像を誤った画像に置き換えない保証 - ネットワーク障害時の明確なエラー表示 - 操作履歴の改竄不能な記録

医療分野では FDA (米国食品医薬品局)、CE (欧州適合性評価)、PMDA (医薬品医療機器総合機構)の承認を必要とする場合があり、ソフトウェアの検証可能性が極めて重要となる。Web エンジンの中で何が起きているかを、ブラウザプロセスから全て観察できる必要がある。これも全層バインディングでなければ実現しにくい。

4.6 教育プラットフォーム

学校向け授業配信、企業研修の e-learning、資格試験の遠隔受験等、「教育コンテンツを Web 技術で配信し、学習者の進捗を管理する」用途である。

この領域の典型的要求：

進捗追跡 - ページ滞在時間の正確な計測 - スクロール位置の記録（読み飛ばし検出） - 動画視聴時間の正確な記録 - クイズ回答時間の計測

不正防止 - 試験中の他ウィンドウ切り替えを検知 - 外部カメラ・マイクによる本人確認 - 画面録画の検知と警告 - コピー・ペーストの制限

学習支援 - ユーザーの入力パターンから学習者の集中度を推定 - 理解度に応じた動的なコンテンツ変更 - 複数学習者の同期（一斉授業時）

運用 - 教員による遠隔操作（画面の強制遷移等） - 授業後の学習履歴レポート - 個別指導の履歴管理

教育プラットフォームでは特に「学習者が正直に学習しているか」を確認する仕組みが重要となる。Web ページ内の JavaScript では取得できない情報（別ウィンドウへのフォーカス移動、システムレベルのスクリーンショット、物理キーボードの入力パターン等）を、ブラウザプロセスから取得する必要がある。これには `ui::FocusManager`、`ui::InputMethod`、OS レベルのイベントフックとの連携が必要となる。

4.7 ロボティクス・ヒューマノイド

産業用ロボット、協働ロボット（コボット）、配送ロボット、自律移動ロボット（AMR）、サービスロボット、ヒューマノイドロボットの分野では、近年 Web 技術を用いた HMI（Human-Machine Interface）と遠隔操作インターフェースが急速に普及している。ROS（Robot Operating System）、ROS 2、NVIDIA Isaac 等のロボティクスフレームワークも、Web ブラウザ経由の可視化ツール（`rviz2` の Web 実装など）を提供する。

この領域の典型的要求：

低遅延テレオペレーション - 遠隔地のロボットからの映像を遅延 100 ms 以下で表示 - オペレータの入力（ジョイスティック、ペダル、ハンドコントローラ）を 20 ms 以下でロボットに送信 - 映像と制御の同期を厳密に管理（時刻同期、タイムスタンプ付与） - 複数カメラ映像（前方、左右、背面、把持部近接）の同時表示と動的レイアウト切替

3D 可視化 - URDF（Unified Robot Description Format）で記述されたロボットモデルのリアルタイム表示 - 点群データ（LiDAR、深度カメラ、Stereo Vision の出力）の可視化 - 占有格子マップ（Occupancy Grid Map）の表示 - SLAM（同時自己位置推定と環境地図作成）結果のオーバーレイ - ロボットの動作計画（軌道、経由点、衝突判定）の可視化

ティーチングペンダント - ロボットアームの手先位置・関節角度の表示と編集 - プログラム作成のためのビジュアルプログラミング環境 - I/O 状態のリアルタイム監視 - エラーコード・アラームの詳細表示と履歴

ヒューマノイド特有の要求 - 二足歩行ロボットの重心・ZMP（Zero Moment Point）可視化 - 全身関節（30〜50 自由度）の状態モニタリング - 対話型 AI との統合（音声入出力、表情認識、感情推定） - 遠隔操作時のテレプレゼンス（VR/AR 統合）

これらの要求のうち、点群表示や URDF ビューアは WebGL / WebGPU を多用する。ヒューマノイドの全身状態表示は百を超えるセンサ値の同期更新を要し、通常の DOM 更新では追いつかない場合がある。`RequestAnimationFrame` と `OffscreenCanvas` を組み合わせた高頻度描画、`SharedArrayBuffer` でのプロセス間データ共有、カスタム IPC 経路での ROS トピック受信など、Web エンジン内部層への介入が実用性を左右する。

産業用ロボットの安全規格（ISO 10218、ISO/TS 15066）や機能安全（IEC 61508）への適合を求められる用途では、表示の正確性と応答性が認証対象となる。ブラウザがフレームを落としたかを検出し、落ちた場合にはロボットを安全停止させる等の仕組みが必要となる。これは `viz::Display` や `cc::LayerTreeHostClient::DidCommit()` の直接監視によって実装される。

4.8 IoT・WoT（Web of Things）

IoT（Internet of Things）は多数のセンサ・アクチュエータ・ゲートウェイ機器から成る分散システムで、UI を持つ機器と持たない機器の両方を含む。W3C の WoT（Web of Things）標準は、これらを Web の語彙（URI、HTTP、JSON-LD）で記述・発見・操作するための枠組みを提供する。Thing Description（TD）、WoT Servient、WoT Scripting API といった概念がこれに属する。

この領域で Web 描画エンジンが活躍するのは、以下のような場面である：

IoT ゲートウェイ機器の HMI

家庭やオフィス、工場に設置される IoT ゲートウェイ（Raspberry Pi、Intel NUC、産業用 PC 上で動作）は、数十〜数百の配下デバイスを管理する。管理画面には次のような要素が含まれる：

- デバイス一覧（オンライン・オフライン状態、最終通信時刻）
- リアルタイムグラフ（温度、湿度、消費電力、センサ値）
- ルールエンジン設定（「温度が 28 度を超えたら空調を強める」等）
- ファームウェア更新の進捗表示
- ログ検索・フィルタ

産業用 HMI（SCADA、DCS）

SCADA（Supervisory Control And Data Acquisition）や DCS（Distributed Control System）の分野では、従来はベンダー独自の GUI が主流だったが、近年は Web 技術で実装するケースが増えている。要求される特性：

- 24 時間 365 日の稼働（長時間運用耐性）
- 複数オペレータによる同時アクセス（ロック機構、変更履歴）
- アラーム音の優先度制御（最重要アラームは必ず鳴らす）
- 冗長構成（プライマリ／セカンダリのフェイルオーバー）
- PLC（プログラマブル・ロジック・コントローラ）との直接通信

WoT Servient としての組み込み機器

W3C WoT 標準では、個々の機器が「Thing」として自己を記述し、他の Thing と相互作用する。小型ディスプレイを持つ WoT Servient では、その Thing Description を自機で表示し、操作インターフェースを提供することがある。例：

- スマート計測器：測定値表示、過去履歴、校正情報
- スマート照明パネル：複数灯具の制御、シーン選択
- ビル管理システム端末：空調・照明・エレベーター統合制御

家電・白物家電の UI

冷蔵庫のドアに付いたタッチパネル、洗濯機のコントロールパネル、炊飯器の LCD、エアコンのリモコン液晶といった「組み込み家電 UI」も、近年は Web 技術で実装するケースが増えている。小型の ARM SoC（RK3368、i.MX 6 クラス）上で動作し、画面サイズは 3〜10 インチ、メモリは 256 MB〜1 GB という制約がある。

これらの用途では、**リソース制約下での動作**が最大の課題となる。通常の Chromium は数百 MB のメモリを消費するが、これを圧縮する必要がある。不要なサービス（Safe Browsing、PDF ビューア、印刷、自動更新等）の無効化、GPU 合成の省略、V8 ヒープサイズの制限等、Chromium 内部への踏み込んだ調整が求められる。

4.9 農業 IT・スマート農業

農業分野の IT 化は、日本国内でも世界的にも進行中の重要領域である。圃場管理、施設園芸の環境制御、畜産モニタリング、精密農業（Precision Agriculture）のいずれも、Web 技術を中核とする統合管理システムが採用されている。

この領域の典型的要求：

圃場管理システム

農家が保有する田畑・果樹園・茶畑等の管理をデジタル化する。GIS（地理情報システム）との統合が不可欠で、次の要素が含まれる：

- 衛星画像（Landsat、Sentinel、Planet）の重畳表示
- ドローン撮影画像のオルソ化・モザイク表示
- NDVI（正規化植生指数）等の植生指標の可視化
- 作物ごとの作付け計画、収穫予測
- 農薬・肥料の散布履歴記録（GAP 認証対応）

これらは大量のラスタ画像と数千〜数万のポリゴン（圃場境界）を同時表示する必要があり、WebGL / WebGPU での高速描画が実用性を左右する。

施設園芸の環境制御

ビニールハウスや植物工場では、温度・湿度・CO2 濃度・光量・養液濃度を精密に制御する。制御画面には次の要素が含まれる：

- 環境データのリアルタイムグラフ（数十のセンサから秒単位で更新）
- 空調・換気・遮光カーテン・灌水ポンプの制御
- 異常検知時のアラート（温度上昇、停電、結露等）
- 植物の生育状態のカメラ監視（AI による自動判定）

農業施設は屋外または半屋外環境にあり、タブレット端末は高温・多湿・粉塵に晒される。ハードウェアはルゲッド化（防塵・防水・耐衝撃）されるが、ソフトウェア側も「常時接続が途切れやすい」「画面が直射日光で見えにくい」といった環境を前提とする必要がある。

ドローン飛行計画・解析

農薬散布ドローン、播種ドローン、測量ドローンの飛行計画と解析も、Web UI で行われることが多い：

- 地図上での飛行経路の編集（ウェイポイント、旋回点、着陸地点）
- 気象データとの統合（風速、降水確率、視界）
- 飛行後の解析結果表示（散布ムラ、撮影範囲のカバレッジ）

畜産モニタリング

酪農・養豚・養鶏施設では、個体管理が重要な課題である。RFID タグ、カメラ、センサからのデータを統合する管理システムが必要となる：

- 個体ごとの体温・活動量・採食量の履歴
- 分娩予兆の検知（AI 行動解析）
- 伝染病兆候の早期発見（体温・行動パターン異常）

農業 IT 特有の要求は「断続的な接続性」である。農村部では LTE 電波が弱い、Wi-Fi が届かない、衛星通信の帯域が限られるといった状況が常態である。Service Worker による完全オフライン動作、ローカルでのデータ集約と定期的な同期、差分アップロードによる帯域節約といった設計が求められる。

4.10 建築・建設技術

建築・土木・建設の分野は、BIM（Building Information Modeling）の普及により、IT 化が急速に進んでいる。設計段階の CAD、施工段階の現場管理、維持管理段階のファシリティマネジメントのすべてで Web 技術の採用が広がっている。

この領域の典型的要求：

BIM ビューア

IFC（Industry Foundation Classes）、glTF、FBX といった 3D モデルフォーマットをブラウザで表示する用途。建築モデルは数百万〜数千万のポリゴン、数十万の建築要素（柱、梁、壁、ドア、窓、設備機器等）を含み、単純な WebGL では描画が困難な規模となる。

要求される機能：

- モデルのストリーミング読み込み（全体ダウンロード前から表示開始）
- LOD（Level of Detail）による動的品質調整
- 建築要素の選択・属性表示（メーカー、型番、納期、コスト）
- 階層表示（フロア単位、系統単位での表示・非表示）
- 断面表示（任意の平面での切断と内部構造確認）
- 寸法計測・面積算定

これらの要求は WebGPU の本領が発揮される領域である。Compute Shader での空間分割、Occlusion Culling、頂点バッファの効率的な管理により、従来 WebGL では不可能だった規模のモデルを扱える。

施工現場タブレット

建設現場では、作業員が持ち運ぶタブレット端末で図面確認、進捗報告、安全点検を行う。要求される特性：

- オフライン動作(現場は Wi-Fi・LTE が不安定)
- 直射日光下での視認性
- 厚手の作業手袋でも操作できる UI
- 粉塵・水濡れ対応(IP65 以上のハードウェア)
- 写真撮影と図面への自動マーキング

施工管理システム

ゼネコンの施工管理では、工程表、協力会社管理、資材発注、品質検査、安全管理を統合したシステムが使われる。要求規模は大きく:

- 数千人の作業員の入退場管理
- 数百の工種を数ヶ月〜数年にわたって管理
- 数万の検査項目の証跡保管
- 何千枚もの図面の版管理

AR 重畳

建設現場での AR(拡張現実)活用も広がっている。タブレットのカメラで撮影した現場映像に、BIM モデルから生成した配管経路、構造躯体、完成予想を重畳表示する。Web エンジンでこれを実現するには、WebRTC(カメラ取得)、WebXR(AR 機能)、WebGPU(3D 描画)の組み合わせが必要となる。

建築分野では、設計から施工、維持管理までのライフサイクルが 50 年以上に及ぶ。特定のソフトウェアベンダーに依存せず、標準化されたフォーマット(IFC、glTF)で長期保存することが重要視される。Web ブラウザはその「標準的ビューア」として機能することが期待されている。

4.11 自動運転・V2X

自動運転車両の開発とその運用には、従来の車載 HMI を大きく超える多様な UI が必要となる。L2(Lane Keeping、ACC)、L3(条件付き自動運転)、L4(特定条件での完全自動運転)、L5(完全自動運転)のそれぞれで、必要な情報表示と操作は異なる。

この領域の典型的要求:

ドライバー HMI（L2〜L3）

- 自動運転システムの作動状況表示（どの機能が有効か）
- 周辺車両・歩行者・自転車の検知結果の視覚化
- 車線変更の予告・実行状況表示
- テイクオーバー（運転権限移譲）要請の明示
- ドライバーの状態監視（目線、眠気、姿勢）

遠隔管制 UI（L4）

無人運転車両の遠隔監視・介入センターで使われる UI。特徴：

- 複数車両の同時監視（地図上の位置、速度、状態）
- 特定車両への映像切替（遅延 100 ms 以下）
- 遠隔介入操作（進路指示、安全停止、手動遠隔運転）
- イベント記録の自動生成（危険接近、急ブレーキ等）

HD マップ表示

自動運転には HD マップ（高精度地図）が必須で、これは車線単位の道路情報を 10 センチメートル単位で記述する。ブラウザ UI での HD マップ表示には：

- 大量ベクタータイル（OpenStreetMap の数百倍のデータ密度）
- レーン、停止線、信号機、標識の 3D レンダリング
- 自車両位置のリアルタイム重畳

V2X 可視化

V2X（Vehicle-to-Everything）は、車両間通信（V2V）、車両・インフラ間通信（V2I）、車両・歩行者間通信（V2P）の総称である。これらの情報を運転者や管制者に示す UI：

- 前方 100 m 先の事故情報を他車から受信して表示

- 信号機のカウントダウン(V2I)
- 歩行者のスマートフォンからの位置情報(V2P)
- 交差点での右折車両への警告(V2V)

機能安全要求

自動運転車載 HMI は、機能安全規格 ISO 26262 への適合が求められる。表示の正確性、応答性、故障時の安全動作が認証対象となり、Web エンジンも「安全関連電子システム」の一部として扱われる場合がある。表示遅延の絶対値、フレームドロップ率の上限、異常時のフェイルセーフ挙動が厳しく規定される。

この分野で Chromium の直接バインディングが必要となる理由は、**応答性の保証**と**異常検出**にある。汎用ブラウザでは「ガベージコレクションが走って一瞬画面が止まる」「タブが増えるとメモリが圧迫される」といった挙動は許容されるが、車載 HMI では許容されない。Compositor のフレーム生成状況の監視、V8 のヒープ状態の監視、レンダラープロセスの応答性監視を、すべてブラウザプロセスから行う必要がある。

4.12 エッジ端末全般に共通する特性

ここまで個別の応用領域を論じてきたが、これらに共通する「エッジ端末」としての特性をまとめる。これらは領域横断的に Web 描画エンジンの採用を検討する際の共通基盤である。

ハードウェア

現代のエッジ端末は、ARM SoC を中核とすることが多い：

- **ハイエンド**: Rockchip RK3588（8 コア、6 TOPS NPU)、NVIDIA Jetson Orin、Qualcomm Snapdragon 8cx
- **ミドル**: NXP i.MX 8M Plus、Allwinner H618、Amlogic A311D
- **ロー**: Rockchip RK3568、Allwinner T507、STMicro STM32MP2
- **超ロー**: ESP32-S3、ESP32-P4（小型ディスプレイ付 IoT 向け）

これらのうち、Chromium が現実的に動作するのはミドル以上(RAM 1 GB 以上、ストレージ 4 GB 以上)である。超ロー領域では、より軽量な埋め込み Web エンジン(QtWebEngine の最小構成、LVGL + embedded JavaScript 等)を検討することになる。

OS 環境

- **Yocto Project**: カスタム組み込み Linux の主流。BSP (Board Support Package)ごとのレイヤーで構成
- **Buildroot**: 軽量なカスタム Linux 構築
- **Android Automotive OS**: 車載特化の Android
- **Tizen**: サムスン系の組み込み Linux
- **独自 RTOS + Linux デュアル**: リアルタイム応答部と UI 部を分離

これらの OS 上で Chromium をビルド・動作させるには、クロスコンパイル環境、適切な sysroot、必要なシステムライブラリのバージョン整合を整える必要がある。

ディスプレイ

- **TFT LCD**: 一般的だが視野角・屋外視認性に制約
- **IPS / OLED**: 高品質表示、コスト増
- **E-Ink**: 電子書籍端末、低消費電力、応答速度制約
- **透明ディスプレイ**: ショーウインドウ、AR のオーバーレイ
- **曲面ディスプレイ**: 車載、医療機器

これらの特性に応じた色補正、リフレッシュレート管理、省電力制御が必要となる。

入力デバイス

- **物理ボタン・ロータリーエンコーダ**: 工業・車載
- **静電容量式タッチ**: 汎用
- **抵抗膜式タッチ**: 手袋対応、低コスト
- **音声入力**: ハンズフリー用途

- **視線入力**: アクセシビリティ、特殊用途
- **ジェスチャ入力**: 非接触操作、医療等

多様な入力デバイスを Web ページに届けるため、バインディング層には柔軟な入力ルーティング機構が必要となる。

5. WebGPU と AI 推論—新しい計算基盤の活用

Chromium の最近の重要な進化として、WebGPU の標準化と実装がある。2023 年の Chrome 113 で安定化した WebGPU は、単なる「WebGL の次世代版」を超え、エッジ計算・AI 推論・科学計算のための汎用並列計算プラットフォームとなる可能性を持つ。本章では、WebGPU とその関連技術が組み込み Web 描画エンジンにもたらす意義を論ずる。

5.1 WebGPU の登場とその意味

WebGPU は W3C の標準仕様で、ネイティブ GPU API（Vulkan、Metal、Direct3D 12）への抽象化レイヤーを Web に提供する。WebGL と比較した主要な改善点：

- **Compute Shader のサポート**: 汎用並列計算が可能（WebGL は描画専用）
- **明示的なパイプライン構築**: 事前コンパイルによる実行時オーバーヘッドの削減
- **Modern GPU 機能の露出**: インダイレクト描画、ストレージバッファ、アトミック操作
- **マルチスレッド**: Compute キューとグラフィックスキューの独立制御
- **バインドグループ**: リソース割り当ての効率化

これらの改善により、WebGPU は従来「ネイティブコードでしかできなかった」並列計算を Web ブラウザ内で実行可能にする。

5.2 AI・機械学習推論のエッジ実行

AI モデルのエッジ推論は、プライバシー（データを外部に送らない）、レイテンシ（通信遅延の排除）、可用性（オフライン動作）、コスト（クラウド料金不要）の四つの観点で重要性が増している。

WebGPU による ML 推論

TensorFlow.js、ONNX Runtime Web、Transformers.js といったライブラリは、WebGPU バックエンドを持つ。これにより次のようなモデルがブラウザ内で実行可能となる：

- **画像認識**: MobileNet（物体分類）、YOLOv8（物体検出）、SAM（セグメンテーション）
- **姿勢推定**: MediaPipe Pose、MoveNet（スポーツ、ヘルスケア、介護見守り）
- **OCR**: TrOCR、PaddleOCR（文書スキャン、帳票読み取り）
- **音声認識**: Whisper（多言語音声認識）、Vosk（軽量版）
- **自然言語処理**: BERT、DistilBERT（テキスト分類、感情分析）
- **小規模 LLM**: TinyLlama、Phi-3-mini、Gemma-2B（オンデバイス対話）

これらモデルの推論速度は、CPU 実行と比較して WebGPU 実行で 5〜50 倍の高速化が観測される。RK3588 の Mali-G610 や、Intel Iris Xe、NVIDIA Jetson の統合 GPU 上で、実用的なレートでの推論が可能となる。

WebNN（Web Neural Network API）

W3C が策定中の WebNN は、ブラウザに「ニューラルネットワーク用の最適化 API」を直接提供する標準である。バックエンドとして WebGPU だけでなく、各プラットフォームの NPU（Neural Processing Unit）や DSP を利用できる：

- Intel の OpenVINO、NPU
- Apple の Core ML、Neural Engine
- Android の NNAPI
- Qualcomm の QNN、Hexagon

Chromium 139 以降で実験的実装が進んでおり、本格的な採用により AI モデルの推論が「特別なライブラリなしに」ブラウザから利用できるようになる。

5.3 組み込み機器での WebGPU 活用パターン

組み込み機器で WebGPU を活用する典型的なパターン：

1. カメラ映像のリアルタイム AI 解析

組み込み機器のカメラ映像（USB カメラ、MIPI CSI カメラ）を、WebGPU 経由で AI モデルに入力し、結果をオーバーレイ表示する。用途例：

- **監視カメラ**: 不審者検出、置き去り物検出、顔認識
- **医療**: 皮膚病変検出、X 線画像解析補助
- **産業**: 製造ラインの外観検査、異物混入検知
- **交通**: ナンバープレート認識、歩行者カウント

WebGPU Texture と WebRTC の MediaStreamTrack を直接連携させることで、CPU への無駄なコピーを避けた高速処理が可能となる。

2. 3D 可視化と計算の統合

BIM、医療画像、流体シミュレーション、分子構造等の大規模 3D データを、描画と計算の両方で WebGPU に統合する：

- **大規模メッシュの動的簡約化**: Compute Shader での LOD 生成
- **ボリュームレンダリング**: 医療 CT / MRI の再構成とレイマーチング
- **流体計算とその可視化**: CFD シミュレーション結果を計算しながら表示
- **パーティクルシステム**: 数百万の粒子をシミュレートして描画

3. 暗号処理・署名検証

WebGPU の Compute Shader を用いた暗号処理も研究されている。組み込み機器の認証処理、ブロックチェーン関連の署名検証、ゼロ知識証明の計算等に利用できる可能性がある。

4. 音声・動画処理

WebCodecs と WebGPU を組み合わせた動画処理：

- リアルタイム動画フィルタ（美肌補正、ノイズ除去）
- コーデック変換（H.264 → AV1）
- 映像からの特徴抽出

5.4 AI 時代における Web 描画エンジンの意義

生成 AI、オンデバイス LLM、マルチモーダル AI の普及により、「エッジで AI を動かす」ことの価値は飛躍的に高まっている。この文脈で Web 描画エンジンに求められる役割：

AI 機能の UI としての役割

AI アプリケーションの多くは、対話型 UI、結果の視覚化、パラメータ調整インターフェースを必要とする。これらを HTML/CSS/JavaScript で実装すれば、プラットフォーム横断で一貫した体験を提供できる。

AI 実行環境としての役割

WebGPU と WebNN の普及により、Web ブラウザ自体が「ユニバーサルな AI 実行環境」となる可能性がある。モデルをダウンロードし、ローカルで実行し、結果をブラウザ内で利用する。この流れが成立すれば、「AI 機能をエッジに配信する」手段として Web エンジンが中心的役割を果たす。

モデル配信・更新のインフラ

AI モデルは定期的な更新を要する。数百 MB〜数 GB のモデルファイルを効率的に配信し、検証し、キャッシュする仕組みとして、Web の既存インフラ（CDN、HTTP/2、Service Worker、Cache API）が適合する。

全層バインディングの重要性の再認識

これらの新しい役割を担うには、Web エンジンの内部への深い介入が必要となる：

- **GPU リソースの直接管理**: 推論モデルのメモリ常駐、プリコンパイル
- **WebGPU バックエンドの選択**: Vulkan か Metal か、特定ドライバの回避
- **計算優先度の制御**: UI 描画と推論計算の資源競合管理
- **外部 AI アクセラレータとの連携**: NPU、DSP への処理委譲
- **モデルのセキュリティ**: 改竄検出、暗号化モデルの復号

これらはいずれも、汎用ブラウザの公開 API では対応しきれない領域であり、組み込み用途に特化した直接バインディングの意義が再確認される。

AI の進展速度を考えると、今後 3〜5 年で「AI 機能を載せない組み込み機器は競争力を失う」状況が到来する可能性が高い。Web 描画エンジンを単なる「UI 表示コンポーネント」ではなく、「AI 実行基盤を含む計算プラットフォーム」として位置づける視座が、これからの設計判断において重要となる。

6. 代替手段との比較

ここまで全層バインディングの技術的意味と応用例を論じてきたが、客観性を保つには代替手段との比較が欠かせない。本章では、前述の 1 章で挙げた選択肢と本アプローチを、複数の観点から比較する。

6.1 機能・性能比較

主要な比較指標として、Web 標準対応率、描画性能、起動時間、メモリ使用量を取り上げる。

Web 標準対応率

方式	エンジン	対応率（2026 年時点）
Chromium 直接バインディング	Blink	98〜100%
CEF / Electron	Blink	98〜100%
Qt WebEngine	Blink	95〜98%（Qt の更新サイクル遅延）
WebKitGTK	WebKit	93〜96%
独立実装 (Servo)	Servo 独自	40〜60%（実用困難レベル）
プラットフォーム WebView	各種	70〜95%（プラットフォーム依存）

ここで押さえておきたいのは、Chromium 系（直接バインディング・CEF・Electron・Qt WebEngine）と WebKit 系（WebKitGTK）のいずれを選んでも、現在の Web 標準には十分に追随できるということである。独立実装はまだ実用域に達しておらず、プラットフォーム WebView は機種依存が大きい。

描画性能（60 fps 達成率）

描画性能は負荷によって大きく異なる。軽量なページでは概ねどの実装も 60 fps 付近を安定して維持する。複雑な CSS アニメーションや多数の合成レイヤーを含むページでは、Chromium 直接バインディング（GPU 合成有効）と Qt WebEngine が 40〜60 fps を維持しやすい。WebKitGTK はその少し下、

30〜55 fps 程度。CEF の OSR モードと、Chromium のソフトウェア合成モードは、高負荷時に 10〜30 fps まで低下することがある。

起動時間とメモリ使用量

空白ページが表示されるまでの起動時間と、常駐メモリ使用量の典型値を軽量な順に並べると、次のようになる。

方式	起動時間	空白表示時メモリ
プラットフォーム WebView	0.1〜0.5 秒	30〜80 MB
WebKitGTK	0.3〜0.8 秒	60〜120 MB
Qt WebEngine	0.5〜1.2 秒	100〜200 MB
Chromium 直接バインディング	0.8〜2.0 秒	100〜200 MB
Electron	1.5〜3.5 秒	150〜300 MB

これらの数値から読み取れるのは、「Chromium 直接バインディングは、機能的には最先端だが、プラットフォーム WebView より重く、WebKitGTK よりやや遅い」という一般傾向である。これは Chromium のマルチプロセスアーキテクチャに起因する部分が大きい。一方、特定用途で設定を最適化すれば、WebKitGTK 並みの性能を引き出すことは可能である。

6.2 保守コスト比較

セキュリティパッチの追随

セキュリティパッチを誰が、どの程度の速度で適用するか、は運用コストと信頼性を大きく左右する。この責任分担は選択する方式によって明確に異なる。

- **プラットフォーム WebView** は最速で、パッチはベンダーが配信し、機器は OS 更新で自動的に適用される（数時間〜数日）

- **WebKitGTK** ではディストリビューション側が配布するパッケージを通じて、数日〜数週間でパッチが届く
- **Qt WebEngine** は Qt Company のリリースを待つ必要があり、数週間〜数ヶ月の遅延がある
- **CEF** も同様に CEF プロジェクトのリリースサイクルに依存する
- **Chromium 直接バインディング**では追随作業は自プロジェクトの責任となり、熟練開発者の継続的関与が必要な代わりに、追随速度は自分で決められる（数日〜数ヶ月）

Chromium は 2〜4 週間に一度セキュリティ修正を含む安定版をリリースする。これを自プロジェクトで追随するには、ソース取得、パッチ移植、ビルド、テスト、リリースという一連の作業が必要となる。この作業には熟練した開発者の時間が継続的に必要であり、軽視できないコストとなる。

アップストリームとの関係

Chromium の API は内部 API であり、「安定している」保証はない。`content::WebContents::Create()` のような基本的な関数シグネチャすら、数年の間に変化する。Chromium 140 から 147 への移行期間で、筆者らが確認した API 変更例を次に示す：

- `RenderViewHost::CreatedRenderView` →`RenderProcessHostObserver::RenderProcessReady` への機能移管
- `ImeTextSpan` のパラメータ追加（Chromium 147）
- `PartitionRoot::DumpStats` のシグネチャ変更（4 引数化）
- `viz::SetSoftwareOutputDeviceEndPaintHook` の削除
- `Wunused-result` 警告の厳格化

これら一つ一つに対応するには、Chromium の該当箇所のコードを読み、自プロジェクトの呼び出し側を修正する必要がある。

6.3 ライセンス・法務比較

主要エンジンのライセンス

- **Chromium** — BSD-3-Clause が主（一部 LGPL、Apache、MIT）。商用利用に問題なし

- **WebKit** — コア部分が LGPL 2.1、WebKit2 関連は BSD。動的リンクを推奨
- **Qt WebEngine** — LGPL 3.0 または商用ライセンス。LGPL では動的リンクを含む制約があり、商用ライセンスで回避可能
- **Servo** — MPL 2.0。条件付きで商用利用可

Chromium は BSD ライクなライセンスが主体で、商用利用が比較的自由である。ただし Chromium の一部構成要素(FFmpeg 等)は LGPL であり、完全な BSD ではない点に注意が必要となる。

コードベースへの改変

直接バインディングのプロジェクトは、Chromium のソースツリーを自プロジェクトに取り込み、必要に応じて改変する。この改変部分のライセンスは、当該ファイルのライセンスに従う。`content/` や `cc/` のファイルは通常 BSD-3-Clause なので、改変版を配布する場合も BSD-3-Clause のライセンス下で配布可能である。

6.4 依存バージョンの制約

プラットフォーム WebView を使う場合、対象プラットフォームのバージョン範囲が自動的に制約となる：

- Android WebView は System WebView の更新状況に依存(古い Android では古い Chromium)
- WKWebView は iOS / macOS のバージョンに連動
- WebView2 は Windows 10 以降、Edge Chromium のインストールが前提

組み込み機器は 10 年以上の長期運用を想定することが多く、プラットフォーム WebView の「勝手な更新」は運用上の問題となりうる。

他方、Chromium 直接バインディングは Chromium のバージョンを自プロジェクトで固定できる。10 年間 Chromium 147 を使い続けることも、技術的には可能である(セキュリティ上は非推奨だが)。これは長期保守プロジェクトにとって重要な性質である。

7. 取り組む際のコストと制約

本章では、全層バインディングに取り組む際に正直に直面するコストと制約を論じる。本文書の冒頭で述べたとおり、本アプローチは「すべての用途に推奨される」わけではない。そうではないことを示すために、本章では誇張なくコストを明記する。

7.1 Chromium 内部 API の不安定性

前述のとおり、`content/`、`cc/`、`viz/`、`ui/` 等の内部 API は公式には「安定 API」ではない。これらはブラウザ内部の実装詳細として位置づけられ、Chromium チームは予告なく変更できる。

実際の変更頻度を例示する：

- `content::WebContentsDelegate` のメソッドシグネチャ: 年 2〜5 回の変更
- `content::NavigationHandle` のメソッド追加・削除: 年 3〜10 回
- `cc::LayerTreeSettings` のフィールド追加・削除: 年 5〜20 回
- Mojo インターフェースの `.mojom` 変更: 年 10〜50 回

これらに追随するため、Chromium を更新する度に十〜百箇所の修正が必要となる。自動化は可能だが、各修正の妥当性は人間の判断を要する。

実例: Chromium 146 から 147 への移行で、筆者らのプロジェクトでは次の修正が必要だった：

- `ImeSetComposition` の引数に `std::vector<ui::ImeTextSpan> suggested_commit_options` が追加（Chromium 147）
- `PartitionRoot::DumpStats()` が 2 引数から 4 引数に変更
- `Wunused-result` 属性が追加された関数の戻り値を明示的に処理する修正（数十箇所）
- ソフトウェアレンダラのフックインターフェース `viz::SetSoftwareOutputDeviceEndPaintHook` の削除と、代替実装

これら修正の累計工数は、二週間程度であった。Chromium は年 8〜10 回の安定版リリースがあるため、単純計算で年間 16〜20 週(約 4 ヶ月分の工数)が追随作業に割かれる。

7.2 ビルドシステムの規模

Chromium のソースツリーは 2026 年時点で以下の規模である：

- **ソースコード行数**: 約 3,500 万行(`content/`、`cc/`、`blink/` 等を含む)
- **ソースツリーサイズ**: 約 40 GB（`git clone --depth=1` 直後）
- **`gclient sync` 後のサイズ**: 約 60〜80 GB（依存ライブラリ含む）
- **ビルド成果物サイズ**: Debug 約 120 GB、Release_Component 約 80 GB
- **オブジェクトファイル数**: 数万個

ビルド時間の目安(筆者らが日常的に使用している環境: AMD Ryzen 9 5950X、RAM 64GB)：

- **フルビルド(Release_Component)**：3 〜 6 時間
- **全モジュールリビルド(設定変更後)**：30 〜 60 分
- **`libblinkgtk.so` 単体リビルド**: 30 秒程度
- **`libcontent.so` リビルド**: 5 〜 15 分

これらは現代的なワークステーションクラスの環境での数値である。より低スペックの環境(ノート PC 等)では更に時間がかかる。

ビルドシステムは GN（Generate Ninja)と Ninja を使う。GN はビルド設定記述のための DSL で、Chromium チームが Make や CMake の代替として開発したものである。GN / Ninja の文法と運用、依存関係の追跡、フラグの扱いには独自の学習が必要となる。

7.3 アップストリーム追随の継続的負担

Chromium は約 4 週間ごとに安定版をリリースする。それぞれのリリース間隔で、次のような作業が必要となる：

リリースサイクルで発生する作業と所要時間の目安

一度の Chromium 更新で、おおよそ次のような作業が必要となる。

- 新バージョンのソース取得(`gclient sync`)に 30 分〜2 時間
- 既存パッチの移植とコンフリクト解決に 2〜8 時間
- API 変更への対応に 2〜16 時間
- ビルドとテスト実行に 4〜12 時間
- 回帰テスト・動作確認に 4〜16 時間
- ドキュメント更新に 1〜4 時間

これらを合計すると、一回の Chromium 更新に 20 時間〜数日が必要となる。これを年 8〜10 回繰り返すのが直接バインディングプロジェクトの実情である。

加えて、Chromium の変更履歴(CHANGELOG)は日々膨大で、自プロジェクトに影響する変更を見落とさずに拾うこと自体が技術である。

7.4 メモリ・ディスクフットプリント

Chromium 由来のランタイムは重い:

ディスク使用量(Release ビルド、Component モード)

- `libcontent.so`: 約 60 MB
- `libblink_core.so`: 約 65 MB
- `libblink_platform.so`: 約 35 MB
- `libblink_modules.so`: 約 45 MB
- `libv8.so`: 約 50 MB
- `libicu.so` (ICU): 約 30 MB
- `libEGL.so` + `libGLESv2.so` (ANGLE): 約 20 MB
- `content_shell.pak`: 約 7 MB
- `v8_context_snapshot.bin`: 約 0.7 MB

- `icudtl.dat`: 約 11 MB
- ロケールファイル: 数 MB

合計で 300〜500 MB、シンボル情報を含めると 1 GB 近くになる。組み込み機器の制約によっては、このフットプリントが許容範囲を超える場合がある。

メモリ使用量

- ブラウザプロセス: 起動直後 30〜60 MB、使用時 80〜150 MB
- レンダラープロセス: 1 ページあたり 50〜200 MB
- GPU プロセス: 30〜80 MB（有効時）
- ユーティリティプロセス: 各 10〜30 MB

複雑なページを表示すると、レンダラープロセスが 500 MB 以上を消費することもある。組み込み用途では `--in-process-gpu` や `--single-process` といったフラグでプロセス数を減らす工夫が可能だが、それぞれにトレードオフがある。

7.5 人的・計算資源

全層バインディングプロジェクトには、次の役割を果たせる人材が必要となる：

1. **Chromium 内部に詳しい開発者**: `content/`、`cc/`、`blink/` のアーキテクチャを理解し、ソースコードを読んで修正できる
2. **UI フレームワーク（GTK、Qt 等）の知識**: バインディング層の設計と実装を担う
3. **ビルドエンジニア**: GN / Ninja、クロスコンパイル、パッケージング（RPM / DEB / tarball）を担う
4. **QA エンジニア**: 回帰テスト、複数プラットフォームでの動作確認、性能測定を担う

小規模なチーム（1〜3 人）では、これら全ての役割を兼ねる必要があり、個人の負担が大きい。筆者の経験では、直接バインディングプロジェクトを継続的に維持するには、最低でも専任 2 人以上の開発者が望ましい。

計算資源としては、開発・ビルド環境として次の程度が必要：

- CPU: 8 コア以上(16 コア以上を推奨)
- RAM: 32 GB 以上(64 GB 以上を推奨)
- SSD: 1 TB 以上(ソースツリーとビルド成果物のため)
- CI/CD: 並列ビルドのためのクラスタ、または多目的ホスト

7.6 テスト・QA の複雑さ

Chromium の変更が自プロジェクトに与える影響を検証するには、包括的なテストが必要となる。一般的な戦略は次のとおり:

- **単体テスト**: バインディング層の各関数の動作確認
- **統合テスト**: WebContents 作成からページロード、入力、描画までの一連の流れ
- **回帰テスト**: 既知のバグ再現パターンの自動検証
- **パフォーマンステスト**: FPS、メモリ、起動時間の測定
- **互換性テスト**: 主要 Web サイト(google.com、wikipedia.org、youtube.com 等)の表示確認

筆者らのプロジェクトでは、ヘッドレス環境(Weston headless)での自動テストと、実 Wayland ディスプレイでの手動目視確認を組み合わせている。自動化できない要素(表示のカクつき、色味、応答性等の主観評価)は、どうしても人間の目で確認する必要がある。

7.7 典型的な落とし穴と教訓

筆者らが実際に遭遇した、全層バインディングに特有の落とし穴を幾つか共有する。これらは設計当初には予見しにくく、実運用で初めて顕在化する類の問題である。

ライブラリ二重ロード問題

Chromium の共有ライブラリ(`libcontent.so`、`libblink_core.so` 等)を、アプリケーションが直接 `dlopen()` し、さらに Chromium 内部でも同じライブラリを使う構成では、メモリ上に同じ `.so` が二度ロードされ、グローバル変数の不整合が生じることがある。特に `cc::g_primary_thread_type` のようなプロセス単位で一意であるべき値が、複数箇所に存在してしまう。解決には `dlopen(..., RTLD_NOW`

| RTLD_GLOBAL）の指定と、ビルド時のシンボル可視性制御（-fvisibility=hidden）の慎重な運用が必要となる。

スレッド Affinity の厳密性

Chromium 内部には「このメソッドはブラウザの UI スレッドからのみ呼ばれる」「このオブジェクトは Compositor スレッドにバインドされている」といった暗黙のスレッド制約が多数存在する。これらは DCHECK_CURRENTLY_ON(BrowserThread::UI) のようなアサーションで検証されるが、アサーションは Debug ビルドでのみ有効で、Release ビルドでは黙って動作してしまうことがある。本番環境で稀に発生するクラッシュが、実はリリース前から存在していたスレッド違反だった、という事態は珍しくない。

Mojo IPC の非同期性

Mojo メッセージは非同期で配送される。ブラウザプロセスから「CollapseSelection() を呼んでテキスト選択を解除」という命令を送っても、実際にレンダラーでその処理が完了するまでには時間がある。この間にユーザーが新しい選択を開始すれば、IPC 完了時には新しい選択を解除してしまう。こうしたレース条件は、単体テストでは検出されず、実機でたまに観察される問題として残りやすい。全層バインディングでは複数層の介入（CSS レベル、JavaScript イベントレベル、キーイベントレベル、IPC レベル）で多層防御することが有効となる。

メモリライフタイムの相互不信

content::WebContents を保持する std::unique_ptr は、どの層が「所有者」か明確にしておかないと、アプリケーション側で delete した後に Chromium 内部のコールバックで参照されてクラッシュすることがある。Chromium は base::WeakPtr と SupportsWeakPtr を多用してこの問題に対処しているが、バインディング層を設計する際にはこれらの仕組みを正しく理解し、寿命管理を慎重に設計する必要がある。

バイナリ再現性

Chromium は GN ビルドフラグ、コンパイラバージョン、リンカオプション、環境変数等、数多くの要素にバイナリが依存する。「昨日ビルドした時は動いていたが、今日 clean してビルドし直すと動かない」といった状況が起こりうる。再現性を保つには、ビルドコンテナの固定（Docker や chroot）、依存バージョンのロック、.gclient や DEPS ファイルの厳密な管理が欠かせない。

これらは Chromium 全層バインディングに限らず、大規模ネイティブソフトウェアに共通する問題でもある。しかし Chromium の場合は規模が桁違いに大きく、一つの問題が顕在化するまで数週間〜数ヶ月かかることもある。運用開始後の数年にわたって継続的に改善していく覚悟が必要となる。

8. 向き不向きの判断基準

前章までを踏まえ、本アプローチがどのような場合に有効で、どのような場合には向かないか、判断基準を整理する。

8.1 向く場合

以下の条件のいずれか（できれば複数）を満たす場合、全層バインディングが有効な選択肢となりうる：

要求仕様の深さ

- 単なる Web ページ表示を超える、プラットフォーム固有の挙動制御が必要
- 描画・入力・セキュリティのいずれかの層で、プラットフォーム WebView や汎用ラッパーでは対応できない要求がある
- レンダリングパイプラインの特定段階での介入（タイミング、ピクセル加工、フレーム同期）が必要

長期運用

- 運用期間が 5 年以上（理想的には 10 年以上）
- プラットフォーム WebView の更新サイクルに合わない運用要件がある
- 運用中のバージョン固定が可能（あるいは必要）な環境

複数プラットフォーム展開

- 複数の組み込み Linux ディストリビューション、カスタム OS を対象とする
- プラットフォーム WebView が存在しない・不十分な環境を含む

専門リソースの確保

- Chromium 内部に精通した開発者が継続的に確保できる

- CI/CD に十分な計算資源が投入できる
- セキュリティパッチの自己対応ができる組織

特殊要求

- DRM、コンテンツ保護、監査、コンプライアンスへの厳格な要求
- 産業・医療・金融等、認証・認可の要求がある領域
- 独自プロトコル、独自スキーム、独自 IPC の実装が必要

8.2 向かない場合

以下の条件に該当する場合、他の選択肢を優先的に検討すべきである：

短期プロジェクト

- 数週間〜数ヶ月のプロトタイピングや実証実験
- 試作機での動作確認が主目的
- 実製品化予定がない、あるいは未定

小規模チーム・単独開発

- 開発者 1 人のみ、かつ Web エンジンの専門知識がない
- 組織的なリソース確保が困難
- 継続的保守のコミットメントが難しい

標準的ブラウザで十分な用途

- ユーザーに標準ブラウザと同等の体験を提供すればよい
- Web ページを表示する以上の特殊要求がない
- プラットフォーム WebView の制限が受容可能

資源制約

- Flash 容量 1 GB 未満の機器
- RAM 256 MB 未満の機器
- CPU が極端に弱い環境(古い ARM Cortex-A5 等)

これらの場合、プラットフォーム WebView や軽量なラッパー(WebKitGTK 等)のほうが合理的な選択となる。

8.3 中間的な場合 —ハイブリッド戦略

「当初はプラットフォーム WebView で始め、要求が明らかになった段階で直接バインディングに移行する」という戦略もありうる。これは次のような形を取る：

1. **Phase 1（プロトタイプ）**：プラットフォーム WebView で基本 UI を実装
2. **Phase 2（検証）**：要求の詳細を把握し、WebView で実現可能か評価
3. **Phase 3（選択）**：実現困難な要求があれば、直接バインディングへの移行を決定
4. **Phase 4（実装）**：バインディング層の設計と実装
5. **Phase 5（安定化）**：継続的な保守体制の構築

このハイブリッド戦略は、早期の投資リスクを抑えつつ、本格化時に必要な基盤を整えるという点で現実的である。筆者らのプロジェクトも、当初は CEF や WebKitGTK の利用を検討した経緯がある。

8.4 判断に際しての問い

本アプローチの採否を決定する際、次の問いに答えることが判断材料となる：

要件側の問い

- 今実装したい機能のうち、どの部分がプラットフォーム WebView の公開 API では実現困難か
- 要求される描画性能(FPS、レイテンシ)は、汎用 Web エンジンの標準動作で満たせるか
- セキュリティ要件(認証、暗号化、改竄防止)のうち、Web エンジン内部との連携が必要なものはあるか
- 運用期間中に必要となる機能追加の見通しは立っているか

組織側の問い

- Chromium のソースを継続的に理解し、パッチを当てられる開発者を確保できるか
- セキュリティインシデント発生時に、24 時間以内にパッチを適用・検証・配布する体制があるか
- ビルドインフラ(CI/CD、ストレージ、計算資源)への投資が現実的か
- ライセンス・著作権について、法務レビューを経た判断ができるか

経済側の問い

- プロジェクトのライフサイクル総コストのうち、Web エンジン層が占める割合はどの程度か
- 代替手段(プラットフォーム WebView、ラッパーライブラリ)で妥協した場合の機会損失はいくらか
- 直接バインディング採用時の初期投資(開発・インフラ)と、継続投資(保守・追随)は資金計画に乗るか

これらの問いに対し「直接バインディングが必要」「体制が組める」「予算が合う」のすべてが肯定的なら、採用の十分条件が揃う。いずれかに不確定要素があれば、まず代替手段で走り、要求が明確化してから移行を検討する方が現実的である。

9. W3C の貢献 —外部実装者の視点から

本文書はこれまで、Chromium という特定の実装を基盤とする全層バインディングについて論じてきた。しかし一歩引いて考えてみれば、そもそもこのような議論が成立する前提として、**Web プラットフォームそのものが標準化され、複数の実装間で相互運用性が保たれていなければならない**。その基盤を四半世紀以上にわたって支え続けてきたのが W3C（World Wide Web Consortium）である。

本章では、Chromium を外から利用・拡張する立場にある実装者として、W3C の貢献がなぜ重要なのかを、できる限り具体的な事例で論ずる。W3C の成果物は、ブラウザ開発者にとってのみ意味を持つのではなく、組み込み Web 描画エンジンの実装者、応用アプリケーションの開発者、そして最終的にはエンドユーザーに至るまで、Web エコシステム全体の価値を下支えしている。

9.1 標準化がなければ全層バインディングは成立しない

本文書で論じてきた「Chromium 全 API バインディング」というアプローチが現実的な選択肢となるのは、Chromium が実装している API の圧倒的多数が **W3C（および密接に連携する WHATWG、IETF、Khronos Group 等）によって標準化された仕様に基づいている**からである。

もし Chromium が Google 独自の非公開プロトコルで動作していたなら、それを基盤とするバインディングプロジェクトは、Google の方針変更に完全に翻弄される。しかし実際には、HTML、CSS、DOM、JavaScript 実行環境（ECMAScript は ECMA が担当するが W3C と協調）、WebGL、WebGPU、WebRTC、Service Worker、Fetch、Streams、WebSocket、WebAuthn、Payment Request、Push API、Notifications、Geolocation、MediaCapture、MediaSource Extensions、Encrypted Media Extensions、WebXR、WebNN、そして Web of Things に至るまで、Chromium が提供する主要機能はすべて W3C 周辺のエコシステムで策定された仕様の実装である。

この事実が意味することは、バインディングプロジェクトが「Chromium 固有の挙動」ではなく「W3C 標準の挙動」にコミットできるということである。Chromium 147 でも Chromium 148 でも、仕様に準拠した動作が期待できる。仮に将来 Chromium が新しいエンジンに置き換わる時代が来たとしても、仕

様が同一である限り、アプリケーションの互換性は保たれる。これは「特定ベンダーへのロックイン」を避ける上で決定的に重要な性質である。

9.2 新しい計算基盤の策定—WebGPU、WebNN、WoT

第 5 章で論じた WebGPU と WebNN は、いずれも W3C の作業部会（Working Group、Community Group）が長年にわたる議論と設計作業の末に結実させた成果である。

WebGPU の策定過程

WebGPU は 2017 年に W3C GPU for the Web Community Group が設立されてから、2023 年の Chrome 113 での安定化まで、約 6 年の歳月を要した。Apple、Google、Mozilla、Microsoft、Intel、NVIDIA、AMD、Samsung といった競合関係にある各社のエンジニアが、共通の仕様を定めるために毎週のように議論を重ねた結果である。Vulkan、Metal、Direct3D 12 という相容れない三つのネイティブ GPU API の共通抽象を作るという、きわめて困難な技術的課題を、ベンダー中立的なプロセスで解決した。

外部実装者の視点からは、この過程はほとんど奇跡的に見える。各社それぞれの既存製品（Metal は macOS/iOS のネイティブ、Direct3D 12 は Windows のネイティブ）との整合性を保ちながら、三者の最大公約数ではなく最小公倍数に近い仕様を策定できた背景には、W3C というニュートラルな場と、技術的な妥当性を競争より上に置く文化がある。

WebNN の意義

WebNN（Web Neural Network API）は 2019 年から W3C Machine Learning for the Web Working Group で議論され、2024 年に Candidate Recommendation 段階に進んだ。AI モデルの推論をブラウザから Vulkan、Metal、DirectML、Core ML、OpenVINO、NNAPI 等のバックエンドに委譲する標準である。

組み込み応用からの期待は大きい。各チップベンダーが独自の NPU ランタイムを提供する現状では、「Qualcomm の NPU を使うための SDK」「Intel の NPU を使うための SDK」「Apple Neural Engine を使うための SDK」と、ハードウェアごとに別々のコードを書かねばならない。WebNN はこれらを Web の層で抽象化し、同じ JavaScript コードが異なるハードウェアで最適に動作することを可能にする。これはエッジ AI の民主化において決定的な役割を果たす可能性がある。

Web of Things（WoT）

第 4.8 節で触れた W3C の Web of Things 標準は、IoT 機器の相互運用性の基盤となる取り組みである。Thing Description（W3C Recommendation、2020 年）、Architecture（2020 年）、Discovery（2023 年）と段階的に整備され、実装例（node-wot 等）も充実してきた。

IoT の世界には MQTT、CoAP、Modbus、BACnet、LoRaWAN、Z-Wave、Zigbee、Matter といった膨大な通信プロトコルと、各ベンダー独自のデータ形式が混在している。WoT は「Thing Description」という共通記述言語でこれらを Web から見える形に抽象化する。組み込み機器の UI を Web 技術で実装する際、WoT を通じて配下機器と対話できるのは、W3C の長年の努力の賜物である。

9.3 国際化と日本語対応—非ラテン文字圏への深い配慮

W3C の成果の中で、日本の技術者として特に感謝すべきは、**国際化（i18n）への長期的コミット**である。W3C の Internationalization Working Group と Japanese Layout Task Force は、CJK（中国語・日本語・韓国語）を含む非ラテン文字圏の組版要求を Web 標準に反映させるため、何年にもわたる活動を続けてきた。

CSS Writing Modes

`writing-mode: vertical-rl` による日本語の縦書き対応は、W3C の CSS Writing Modes Level 3（2019 年 Recommendation）の成果である。これにより、日本の小説、新聞、学術論文、教科書、漫画といった伝統的な縦書き組版を Web ブラウザで正確に表現できるようになった。現在も Level 4 で改良が続いている。

ルビ（振り仮名）

`<ruby>` 要素と関連する CSS は、HTML5（W3C Recommendation、2014 年）で標準化された。日本語教材、漢字学習アプリ、古典文学のデジタル化、そして外国人向け日本語学習コンテンツで、振り仮名を正確にレンダリングできる基盤となっている。

禁則処理

`line-break` プロパティは、日本語の句読点がページ頭に来ない、長音符が行頭に孤立しないといった禁則処理を制御する。これは『日本語組版処理の要件』(JLReq、W3C Working Group Note)を土台とする成果で、日本の伝統的組版慣習を Web に継承する努力の結晶である。

その他の日本語対応

- フォントサイズ調整(`font-size-adjust`)
- 文字方向(`direction`、`unicode-bidi`)
- 文字区切り(`word-break`、`overflow-wrap`)
- Emoji の標準化(Unicode 協会と連携)

これらのいずれも、「日本の組版文化は守られるべきだ」という価値観が W3C 内部で共有され、該当する英語話者主体ではないコミュニティが仕様策定に参画できる仕組みがあったからこそ実現したものである。電子書籍端末、教科書ビューア、新聞アプリが Web 技術で実装できる今日の状況は、決して当然ではなく、W3C の継続的な努力によって可能になった。

9.4 相互運用性と Web Platform Tests

W3C は仕様を策定するだけでなく、**複数の実装が同じ仕様を同じように解釈していることを検証する**仕組みも整備してきた。Web Platform Tests(WPT、wpt.fyi)は、その中核をなすテスト基盤である。

2026 年時点で WPT は 200 万件を超えるテストケースを持ち、Chromium、Firefox、Safari(WebKit)、Servo といった主要エンジンが日常的にテストを走らせている。各エンジンの WPT パス率は公開ダッシュボードで確認でき、仕様違反があればエンジン側の修正を促す圧力となる。

Interop イニシアチブ

2021 年から始まった Interop プロジェクト(旧 Compat)は、毎年 Web プラットフォームで「特に互換性向上が望まれる機能」を投票で選定し、主要ブラウザベンダーがそれらの仕様準拠を重点的に改善する取り組みである。Interop 2023 では Container Queries、Color Functions、CSS Nesting、Shadow DOM 等が対象となり、いずれも大幅な互換性改善が達成された。Interop 2024 / 2025 も継続中である。

外部実装者にとって、この相互運用性の高まりは極めて重要である。ある Web ページが「Chromium では正しく表示されるが WebKit では崩れる」という状況は、過去 20 年で劇的に減少した。Chromium を採用しても、WebKit を採用しても、作ったコンテンツが広く動作する保証が、W3C と各ベンダーの協調によって下支えされている。

9.5 アクセシビリティへの長期的コミット

W3C の Web Accessibility Initiative（WAI）は、1997 年の設立から四半世紀以上にわたって、Web のアクセシビリティを推進してきた。

WCAG（Web Content Accessibility Guidelines）

WCAG 2.0（2008 年）、2.1（2018 年）、2.2（2023 年）と継続的に改訂され、Web コンテンツが視覚・聴覚・運動機能・認知に制約のある利用者にも利用できるようにするための指針を提供している。各国政府（米国 ADA、欧州 EAA、日本の JIS X 8341）が法的要件として WCAG を参照しており、アクセシビリティは単なる「良い実践」ではなく法的要求となりつつある。

WAI-ARIA

Accessible Rich Internet Applications（ARIA）は、動的な Web アプリケーションが支援技術（スクリーンリーダー等）と正しく連携するための仕様である。`role` 属性、`aria-*` 属性を通じて、視覚的表現と意味的構造を分離する。

組み込みアプリケーションでの意義

医療機器、公共交通機関の券売機、ATM、選挙投票端末、教育端末といった公共性の高い組み込み機器では、アクセシビリティ対応が法的義務である。Web 技術で UI を実装することで、ARIA、WCAG、`aria-live`、キーボードナビゲーションといった豊富なアクセシビリティ機能を最初から享受できる。これはネイティブ UI をゼロから実装する場合には大変な工数を要する部分である。

9.6 開放性・多様性・長期性というプロセスの価値

W3C の仕様策定プロセスそのものにも、外部実装者が敬意を表すべき特性がある。

開放性

W3C の作業部会はほぼすべて GitHub 上で公開の issue tracker を持ち、誰でも仕様案にコメントできる。策定中の仕様は公開ドラフトとして読める。商用ベンダーでなくとも、個人研究者でも、スタートアップ企業でも、議論に参加できる。これは「仕様は限られた人たちの密室で作られる」という誤解を事実で否定する。

ベンダー中立性

W3C メンバー企業は競合関係にあることが多いが、仕様策定の場では「技術的妥当性」が優先される。特定ベンダーに有利な設計は批判される。これは 1990 年代のブラウザ戦争の反省から生まれた文化であり、今日も大切に守られている。

長期視座

W3C の仕様は、一度標準化されると何十年と維持されることを前提に設計される。HTML、HTTP、URL はもう 30 年以上の歴史を持ち、今も現役で動作する。後方互換性の維持は、W3C の最も保守的で、かつ最も価値ある原則のひとつである。組み込み機器が 20 年稼働するという現実の前提と、Web の長寿命設計は本質的に相性がよい。

知的財産権ポリシー

W3C は Royalty-Free ライセンス方針を堅持しており、仕様に含まれる技術に対して参加者は特許権行使を行わないことを宣言する。これにより、Web 標準を実装する誰もが、将来の特許訴訟を恐れずに開発できる。MPEG-LA 方式の特許プールに依拠するビデオコーデックの世界との対比で見ると、この方針の価値は格別である。

9.7 感謝と、これからへの期待

本文書の執筆者は一介の外部実装者に過ぎず、W3C の仕様策定に直接貢献してきたわけではない。しかし、本文書で論じたすべての技術的選択肢— 電子書籍端末のコピー保護から、ロボティクスの 3D 可視化、農業 IT の圃場管理、医療画像の色精度、自動運転の HMI、エッジ AI の推論基盤に至るまで — は、W3C の成果物なしには存在しえない。

電子書籍で縦書きが正しく表示される時、ロボットの状態表示が複数ブラウザで同じように動く時、工場の SCADA 画面がタブレットでも PC でも同じ操作感で使える時、農家が圃場管理アプリを異なる端末で開ける時、その背後には W3C とその貢献者たちの累積的な努力がある。

特に感謝を述べたい点を三つ挙げる。

一つ目は、新しい計算基盤への道筋を開いてきたこと。WebGPU、WebNN、WebXR、Web of Things、WebCodecs のような「単なる Web ページ表示」を超える基盤の策定は、組み込みや特殊用途の Web エンジン活用を現実的にした。これらの仕様策定に費やされた膨大な時間と労力に深く感謝する。

二つ目は、日本語を含む非ラテン文字圏の組版要求に真摯に向き合ってきたこと。Writing Modes、Ruby、JLReq、禁則処理、縦中横といった日本語組版特有の要求が Web 標準に取り込まれたのは、W3C 内部で日本人技術者を含む多言語コミュニティが活動した結果である。日本発の技術文化が Web の普遍的インフラに反映されていることは、日本の技術者として誇らしく、また感謝すべきである。

三つ目は、長期的視座と後方互換性を守り抜いてきたこと。短期的な流行に流されず、30 年前に書かれた HTML が今日も動く、20 年前の HTTP 実装が今も通信できる、という普遍性を維持することの困難さは、組み込み機器を 10 年以上保守した経験のある者ほど痛感する。この普遍性は、Web 標準を組み込み用途の基盤として選ぶ最大の理由のひとつである。

Web プラットフォームはこれからも進化を続け、新しい応用領域が現れる。AI、ロボティクス、自律運転、拡張現実、量子計算といった技術の進展に応じて、新しい W3C 仕様が策定されるであろう。外部実装者として、私たちはその成果を享受し、また可能な範囲で検証結果や実装経験のフィードバックを返すことで、この循環に少しでも貢献したい。

本文書が、W3C の貢献がなぜ Web 技術の応用領域拡大の土台となっているのかを、組み込み実装者の視点から提示する一助となれば幸いである。

10. 結び

本文書では、Chromium 全 API バインディングという選択が持つ技術的意味と、組み込み Web 描画エンジンにおける意義を論じた。要点を再整理する：

技術的意味 1. 半透過抽象を回避し、Web エンジン内部のあらゆる層に介入可能とする 2. 組み込み用途固有の要求に対するカスタマイズ余地を確保する 3. パフォーマンス最適化の自由度を最大化する 4. Web プラットフォーム最新機能への即時アクセスを可能にする 5. プロセス間通信経路への直接的な拡張を許容する

応用価値（拡大し続ける領域） 1. 電子書籍・教科書端末における精緻なコンテンツ保護 2. キオスク・デジタルサイネージにおける操作制限と固定化 3. 車載 HMI・産業機器における応答性とリソース最適化 4. 金融取引端末における低遅延と整合性保証 5. 医療画像ビューアにおける色精度と認証対応 6. 教育プラットフォームにおける学習追跡と不正防止 7. ロボティクス・ヒューマノイドにおける 3D 可視化と低遅延遠隔操作 8. IoT ゲートウェイ・WoT Servient における軽量長時間稼働 9. スマート農業における屋外視認性と断続的接続への耐性 10. BIM ビューア・施工管理における大規模 3D モデルの効率的描画 11. 自動運転車両における機能安全と HD マップ処理 12. WebGPU を基盤とするオンデバイス AI 推論の UI 統合

現実的コスト 1. Chromium 内部 API の不安定性への継続的対応 2. 40 GB を超えるソースツリーと数時間の初期ビルド時間 3. 年 8〜10 回の Chromium 更新への追随工数 4. 300〜500 MB のランタイムフットプリント 5. 専任 2 人以上を推奨する継続的保守体制

これらを総合すれば、本アプローチは「万能の解決策」では決してない。むしろ、**特定の用途において、他の選択肢では満たせない要求を満たすための手段**である。そしてその「特定の用途」には、前章で列挙したような広範な応用領域が含まれる。

Web 技術は組み込み機器の UI 実装手段として、今後さらに広がっていくと予想される。HTML / CSS の宣言的な記述、JavaScript の柔軟性、豊富な開発ツール、国際化対応の容易さは、ネイティブ UI 開発では得難い利点である。この利点を享受しつつ、組み込み固有の要求にも応える実装基盤の一つとして、Chromium 全 API バインディングは現実的な選択肢となる。

これからの展望

執筆時点(2026 年)で進行中の重要な潮流として、次のものが挙げられる：

- **WebGPU と WebNN による AI 基盤化**: Web エンジンが単なる UI プラットフォームから、エッジ AI の実行基盤へと役割を拡張しつつある。これはロボティクス、農業 IT、医療、自動運転のいずれでも価値を持つ
- **Wayland と Linux 組み込みの成熟**: X11 を排し Wayland をネイティブとする組み込み Linux 環境が主流となり、Chromium もこれに歩調を合わせている
- **WoT 標準の実装進展**: W3C の Web of Things が IoT 機器の相互運用性の基盤となり、Web エンジンはその「共通言語」として機能する
- **長寿命サイクルへの適合**: 組み込み機器は 10 年〜30 年の運用を前提とする一方、Chromium は 4 週間サイクルで更新される。両者の時間軸の違いをどう折り合わせるかが、今後の中心的課題となる
- **エッジ・クラウド協調**: すべての計算をクラウドに集約するのでもなく、すべてをエッジで行うのでもなく、動的に分担する設計。Web 技術はこの協調を表現する自然な手段である

これらの潮流に照らすと、Web 描画エンジンの「全層バインディング」という選択は、単なる技術的こだわりではなく、**急速に拡大する応用領域の要求に応え続けるための実装戦略**として位置づけられる。一つのエンジンが電子書籍、医療、自動運転、ロボティクス、農業、建設、教育、IoT のすべてで使われるとき、それぞれの領域固有の要求に柔軟に対応できる基盤こそが価値を持つ。

そして本文書が繰り返し強調したいのは、こうした多様な応用領域の拡大が可能となっている最大の理由は、**W3C を中心とする標準化コミュニティが築き上げてきた Web プラットフォームの普遍性・相互運用性・長期安定性**にあるという事実である。本文書が論じる全層バインディングは、あくまで「W3C 標準に基づく Web プラットフォームを、特定の組み込み環境に適用する実装手法」に過ぎない。根本にある価値の源泉は、実装ではなく標準そのものである。

本文書が、組み込み Web 描画エンジンの採用・設計・実装を検討する技術者にとって、手段選択の判断材料となれば幸いである。Web プラットフォームはこれからも進化を続け、新しい応用領域が継続的に現れる。一度きりの判断ではなく、要求と技術の両方が変化する前提のもとで、柔軟に設計を見直してい

く姿勢が重要となる。そしてその変化の只中で、W3C の標準化プロセスが引き続き開放性と長期視座を保ち続けることを、一人の外部実装者として心より期待している。

参考文献・リソース

Chromium プロジェクト公式資料

- Chromium Project Home. https://www.chromium.org/
- Chromium Source Code. https://source.chromium.org/chromium
- Chromium Design Documents. https://www.chromium.org/developers/design-documents/
- How Blink works. https://docs.google.com/document/d/1aitSOucL0VHZa9Z2vbRJSyAIsAz24kX8LF-ByQ5xQnUg/
- Mojo IPC. https://chromium.googlesource.com/chromium/src/+/main/mojo/README.md

代替実装のドキュメント

- Chromium Embedded Framework (CEF). https://bitbucket.org/chromiumembedded/cef/
- Electron. https://www.electronjs.org/
- WebKitGTK. https://webkitgtk.org/
- Qt WebEngine. https://doc.qt.io/qt-6/qtwebengine-index.html
- Servo Project. https://servo.org/
- Ladybird Browser. https://ladybird.org/

Web 標準化団体と仕様リソース

- **W3C（World Wide Web Consortium）**: https://www.w3.org/
- **WHATWG HTML Standard**: https://html.spec.whatwg.org/
- **Web Platform Tests (WPT)**: https://wpt.fyi/
- **Interop イニシアチブ**: https://wpt.fyi/interop-2024
- **IETF HTTP Working Group**: https://httpwg.org/

W3C 主要仕様（本稿で参照した新世代技術）

- **WebGPU Specification**: https://www.w3.org/TR/webgpu/
- **Web Neural Network API (WebNN)**: https://www.w3.org/TR/webnn/
- **Web of Things (WoT) Architecture**: https://www.w3.org/TR/wot-architecture/
- **Web of Things (WoT) Thing Description**: https://www.w3.org/TR/wot-thing-description/
- **WebXR Device API**: https://www.w3.org/TR/webxr/
- **WebCodecs**: https://www.w3.org/TR/webcodecs/
- **Service Workers**: https://www.w3.org/TR/service-workers/

W3C 国際化 / 日本語組版

- **W3C Internationalization Working Group**: https://www.w3.org/International/
- **CSS Writing Modes Level 3 / 4**: https://www.w3.org/TR/css-writing-modes-3/
- **日本語組版処理の要件（JLReq）**: https://www.w3.org/TR/jlreq/
- **Ruby Annotation**: https://www.w3.org/TR/ruby/

W3C アクセシビリティ

- **Web Accessibility Initiative (WAI)**: https://www.w3.org/WAI/
- **Web Content Accessibility Guidelines (WCAG) 2.2**: https://www.w3.org/TR/WCAG22/
- **WAI-ARIA**: https://www.w3.org/TR/wai-aria-1.2/

W3C プロセス

- **W3C Patent Policy (Royalty-Free)**: https://www.w3.org/Consortium/Patent-Policy/
- **W3C Process Document**: https://www.w3.org/Consortium/Process/

組み込み Linux 関連

- Yocto Project. https://www.yoctoproject.org/
- Automotive Grade Linux. https://www.automotivelinux.org/

- Wayland Protocol Specification. https://wayland.freedesktop.org/

本文書の位置づけ

本文書は、BlinkGTK Development Team が Chromium 全 API バインディングに取り組んできた経験に基づいて執筆された。具体的な実装プロジェクトへの言及は最小限にとどめ、読者が自身のプロジェクトで判断を行う際の参考資料となることを意図している。特定の製品・プロジェクトを推奨・否定する意図はない。

Interop Tokyo 2026 での配布

本文書は Interop Tokyo 2026（2026 年 6 月、幕張メッセ）において、会場設置の QR コード経由でダウンロード配布される。Interop Tokyo は 1994 年の Interop Japan 発足以来、日本のネットワーク・インフラ技術の一大拠点として機能してきた。その参加者— ネットワークエンジニア、クラウドアーキテクト、IoT 技術者、セキュリティ専門家、キャリア通信事業者、SIer 各位 —にとって、本文書が組み込み Web 描画エンジンの視座を提供し、インフラ層と Web プラットフォーム層の接点を考える一助となれば幸いである。

Interop ShowNet への敬意

本文書の執筆者は ShowNet の運営に直接携わった経験はないが、四半世紀以上にわたって多ベンダー相互接続性の実証を続けてきた ShowNet NOC の皆様の努力に、一技術者として深い敬意を表する。「動くことを現場で示す」というその姿勢は、仕様策定と実装検証を両輪とする W3C の哲学と根底で繋がっている。ネットワーク層と Web プラットフォーム層は別々のレイヤーにあるが、標準化に基づく相互運用性こそが技術の民主化と長期的価値創出の源であるという信念は共通している。

ライセンス

本文書は Creative Commons Attribution 4.0 International (CC BY 4.0) ライセンス下で公開される。著作権者のクレジットを明示する限り、複製・配布・翻案が自由である。Interop Tokyo 2026 の参加者は、本文書を社内共有・引用・翻訳等の目的で自由に利用できる。

改訂履歴

- 2026-04-17: 初版(Interop Tokyo 2026 配布版)